Lecture 6. Space Complexity

Spring 2026, *Theory of Computation*, IIIS, Tsinghua

Time Complexity

- Let M be a *deterministic* Turing machine that halts on all inputs. The running time or time complexity of M is the function f: N→N, where f(n) is the maximum number of steps that M uses on any input of length n



Time Complexity

- Let M be a *deterministic* Turing machine that halts on all inputs. The running time or time complexity of M is the function f: N→N, where f(n) is the *maximum number* of steps that M uses on any input of length n

Worst-case analysis



Time complexity class

- TIME(t(n)): all languages that are decidable by an O(t(n)) time *deterministic* Turing machine.
- NTIME(t(n)): all languages that are decidable by an O(t(n)) time *nondeterministic* Turing machine.
  - The running time of a nondeterministic Turing machine is the maximum number of steps that it uses on any branch of the computation on any input of length n



Time complexity

$$P = \bigcup_k TIME(n^k)$$

$$NP = \bigcup_k NTIME(n^k)$$

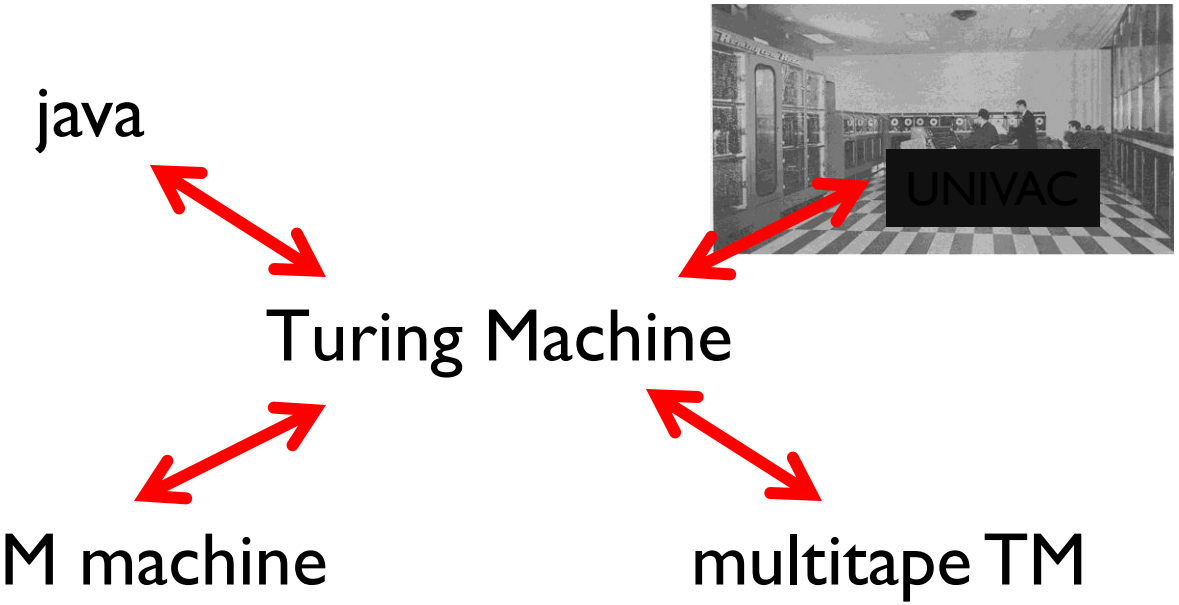
$$EXP = \bigcup_k TIME(2^{n^k})$$ or EXPTIME

$$NEXP = \bigcup_k NTIME(2^{n^k})$$



Efficiency and the Church-Turing thesis

- The Church-Turing thesis says all these models are equivalent in power…



… but not in running time!



The Cobham-Edmonds thesis

- However, there is an extension to the Church-Turing thesis that says

For any realistic models of computation M_1 and M_2:

> M_1 can be simulated on M_2 with at most polynomial slowdown

- So any task that takes time T on M_1 can be done in time (say) T^2 or T^3 on M_2



The Cobham-Edmonds thesis

- However, there is an extension to the Church-Turing thesis that says

For any realistic models of computation M_1 and M_2:

M_1 can be simulated on M_2 with at most polynomial slowdown

- So any task that takes time T on M_1 can be done in time (say) T^2 or T^3 on M_2

Quantum computation does NOT change the Church-Turing thesis, that is, what is computable. But it does seem to change what is computable in polynomial time



Space Complexity

- Complexity classes correspond to bounds on resources

- One such resource is space: the number of tape cells a TM uses when solving a problem





Space Complexity Classes

For any function $f:N \rightarrow N$, we define:

$SPACE(f(n))=\{ L : L$ is decidable by deterministic TM M which scans $O(f(n))$ tape cells$\}$

$NSPACE(f(n))=\{ L : L$ is decidable by non-deterministic TM M which scans $O(f(n))$ tape cells on any branch of its computation$\}$



Example

- SAT∈SPACE(n)

- Although SAT is NP-complete, it can be solved in O(n) space

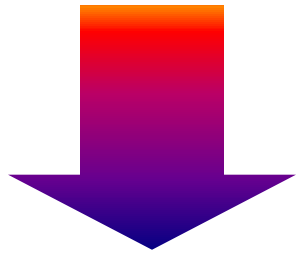
  - e.g. $\phi=(x_1 \vee x_2) \wedge (\sim x_2 \vee x_3 \vee \sim x_4) \wedge (\sim x_1)$
  - For each variable, assign true or false, check satisfiability
  - Takes linear space



Low Space Classes

Definitions (logarithmic space classes):

- L = SPACE(log n)
- NL = NSPACE(log n)





Low Space Classes

Definitions (logarithmic space classes):

- L = SPACE(log n)
- NL = NSPACE(log n)

- PSPACE=U_k SPACE(n^k)
- NPSPACE=U_k NSPACE(n^k)
  - for constant k



Low Space Classes

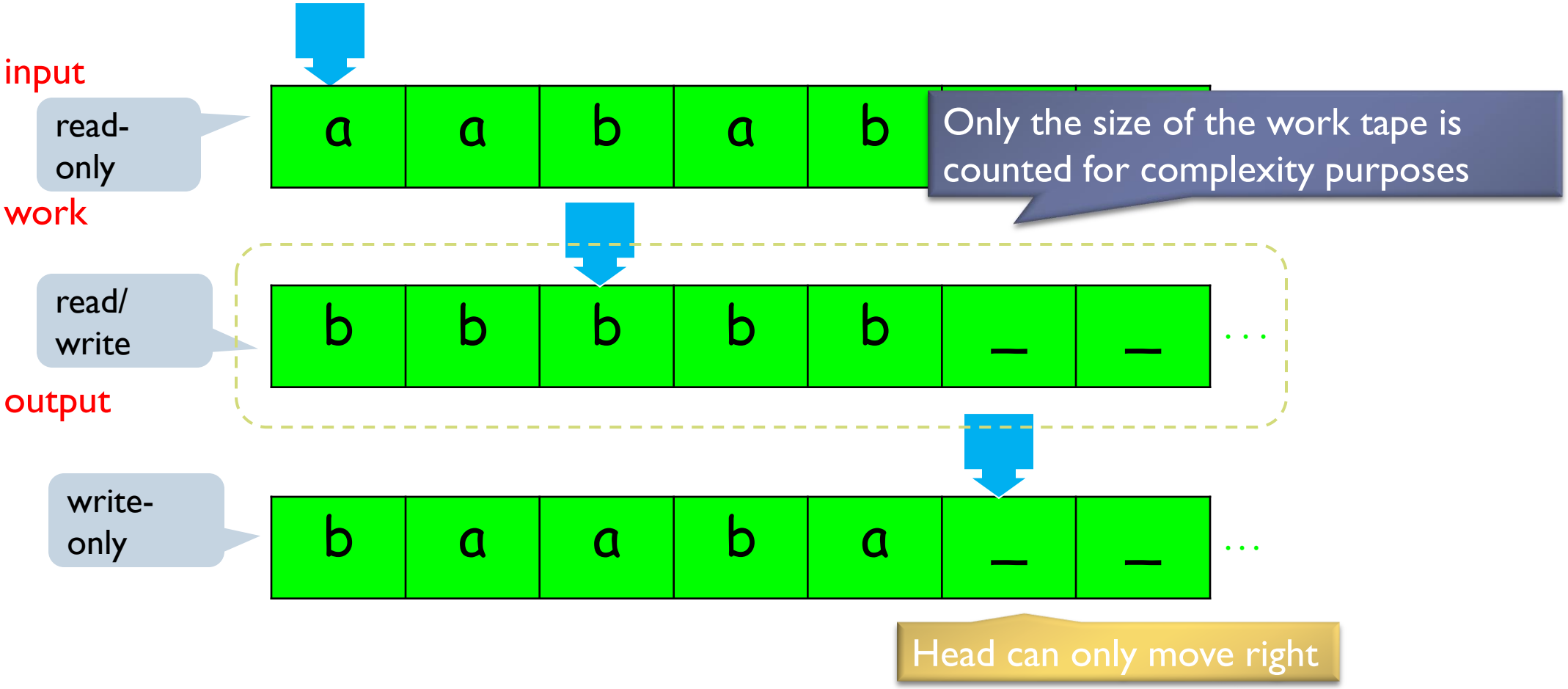
Definitions (logarithmic space classes):

- L = SPACE(log n)
- NL = NSPACE(log n)

- How can a TM use only log n space if the input itself takes n cells?!



3Tape Machines



Sublinear space

- Your memory is much smaller than the amount of information
  - Sublinear space algorithm allow the computer to manipulate the data without storing all of it in its memory





Configurations

Which objects determine the configuration of a TM of the new type?

- The content of the work tape
- The machine's state
- The head position on the input tape
- The head position on the work tape
- The head position on the output tape

If the TM uses logarithmic space, there are polynomially many configurations



Configurations

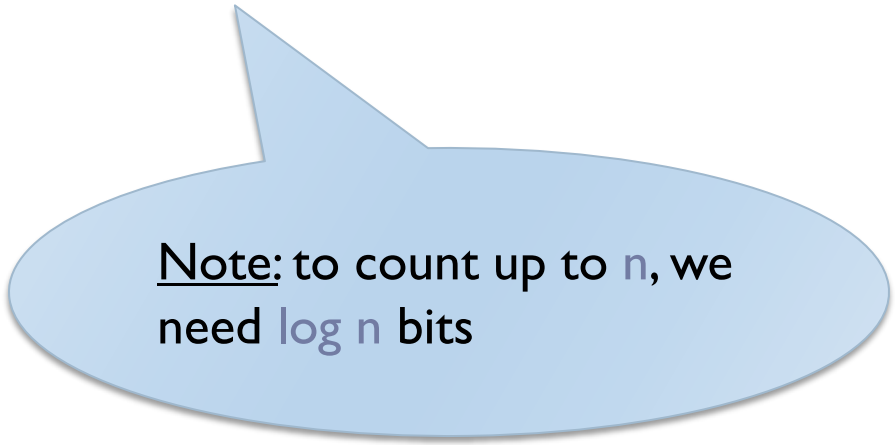
Which objects determine the configuration of a TM of the new type?

- The content of the work tape
- The machine's state
- The head position on the input tape
  - we need O(log n) space to record the position
- The head position on the work tape
- The head position on the output tape
  - write-only: so we don't need to record the position



Example

Question: How much space would a TM that decides $\{a^n b^n \mid n>0\}$ require?





Simple Observation (Any model)

- $\mathrm{TIME}(f(n)) \subseteq \mathrm{SPACE}(f(n))$

- $\mathrm{SPACE}(f(n)) \subseteq \mathrm{TIME}(2^{O(f(n))})$
  - *when $f(n) = \Omega(\log n)$*



Simple Observation (Any model)

- $TIME(f(n)) \subseteq SPACE(f(n))$
  - a machine that uses f(n) time can use at most f(n) space

- $SPACE(f(n)) \subseteq TIME(2^{O(f(n))})$
  - a machine uses O(f(n)) space can have at most $2^{O(f(n))}$ configurations
  - a TM that halts may not repeat a configuration
  - so the total time to "list" all configurations is

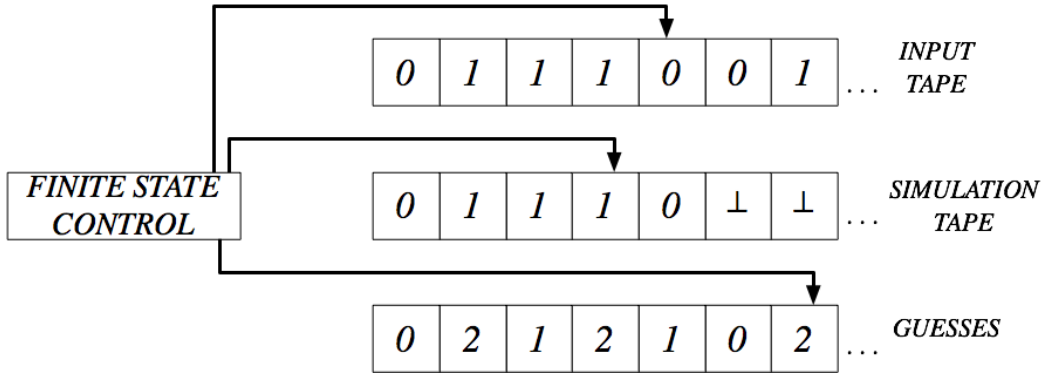
$$f(n) \cdot 2^{O(f(n))} = 2^{O(f(n))}$$

- what about NTIME and NSPACE ?



Simple Observation

- $TIME(f(n)) \subseteq SPACE(f(n))$
- $SPACE(f(n)) \subseteq TIME(2^{O(f(n))})$
- $NTIME(f(n)) \subseteq SPACE(f(n))$
  - we can enumerate all possible $O(f(n))$ "guesses" in $O(f(n))$ space
  - size of "guesses tape" is $O(f(n))$





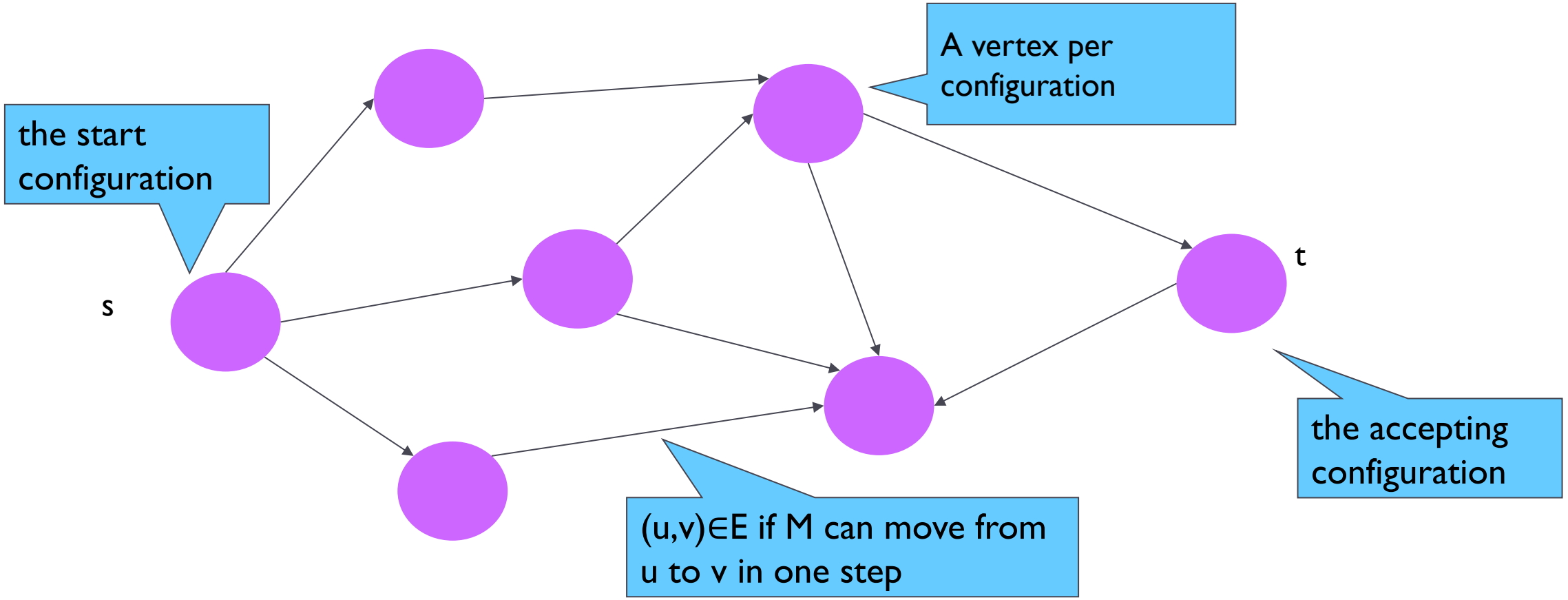
Simple Observation

- $\mathrm{TIME}(f(n)) \subseteq \mathrm{SPACE}(f(n))$
- $\mathrm{SPACE}(f(n)) \subseteq \mathrm{TIME}(2^{O(f(n))})$
- $\mathrm{NTIME}(f(n)) \subseteq \mathrm{SPACE}(f(n))$
- $\mathrm{NSPACE}(f(n)) \subseteq \mathrm{TIME}(2^{O(f(n))})$



Configurations Graph

A Computation of a NTM M on an input x can be described by a graph $G_{M,x}$:



Simple Observation

- $\mathrm{TIME}(f(n)) \subseteq \mathrm{SPACE}(f(n))$
- $\mathrm{SPACE}(f(n)) \subseteq \mathrm{TIME}(2^{O(f(n))})$
- $\mathrm{NTIME}(f(n)) \subseteq \mathrm{SPACE}(f(n))$
- $\mathrm{NSPACE}(f(n)) \subseteq \mathrm{TIME}(2^{O(f(n))})$
  - Construct the configuration graph with $2^{O(f(n))}$ vertices
  - Check whether accepting configurations are reachable from start configuration



Simple Observation

- $TIME(f(n)) \subseteq SPACE(f(n))$
- $SPACE(f(n)) \subseteq TIME(2^{O(f(n))})$
- $NTIME(f(n)) \subseteq SPACE(f(n))$
- $NSPACE(f(n)) \subseteq TIME(2^{O(f(n))})$

$$L \subseteq NL \subseteq P \subseteq NP \subseteq PSPACE \subseteq NPSPACE \subseteq EXPTIME$$



Simple Observation

- $TIME(f(n)) \subseteq SPACE(f(n))$
- $SPACE(f(n)) \subseteq TIME(2^{O(f(n))})$
- $NTIME(f(n)) \subseteq SPACE(f(n))$
- $NSPACE(f(n)) \subseteq TIME(2^{O(f(n))})$

$$L \subseteq NL \subseteq P \subseteq NP \subseteq PSPACE = NPSPACE \subseteq EXPTIME$$

we will see why later



Simple statements

- $L \subseteq P$
  - The number of possible configurations: $2^{O(\log n)}$
- $L \subseteq NL$
- Define $L^2 = SPACE(\log^2 n)$
- What about L^2 and NL?

- We'll first see NL-completeness



Graph Connectivity

PATH (st-connectivity)

- Instance: a directed graph $G=(V,E)$ and two vertices $s,t \in V$
- Problem: To decide if there is a path from s to t in G?



Graph Connectivity

PATH (st-connectivity)

- Instance: a directed graph $G=(V,E)$ and two vertices $s,t \in V$
- Problem: To decide if there is a path from s to t in G?

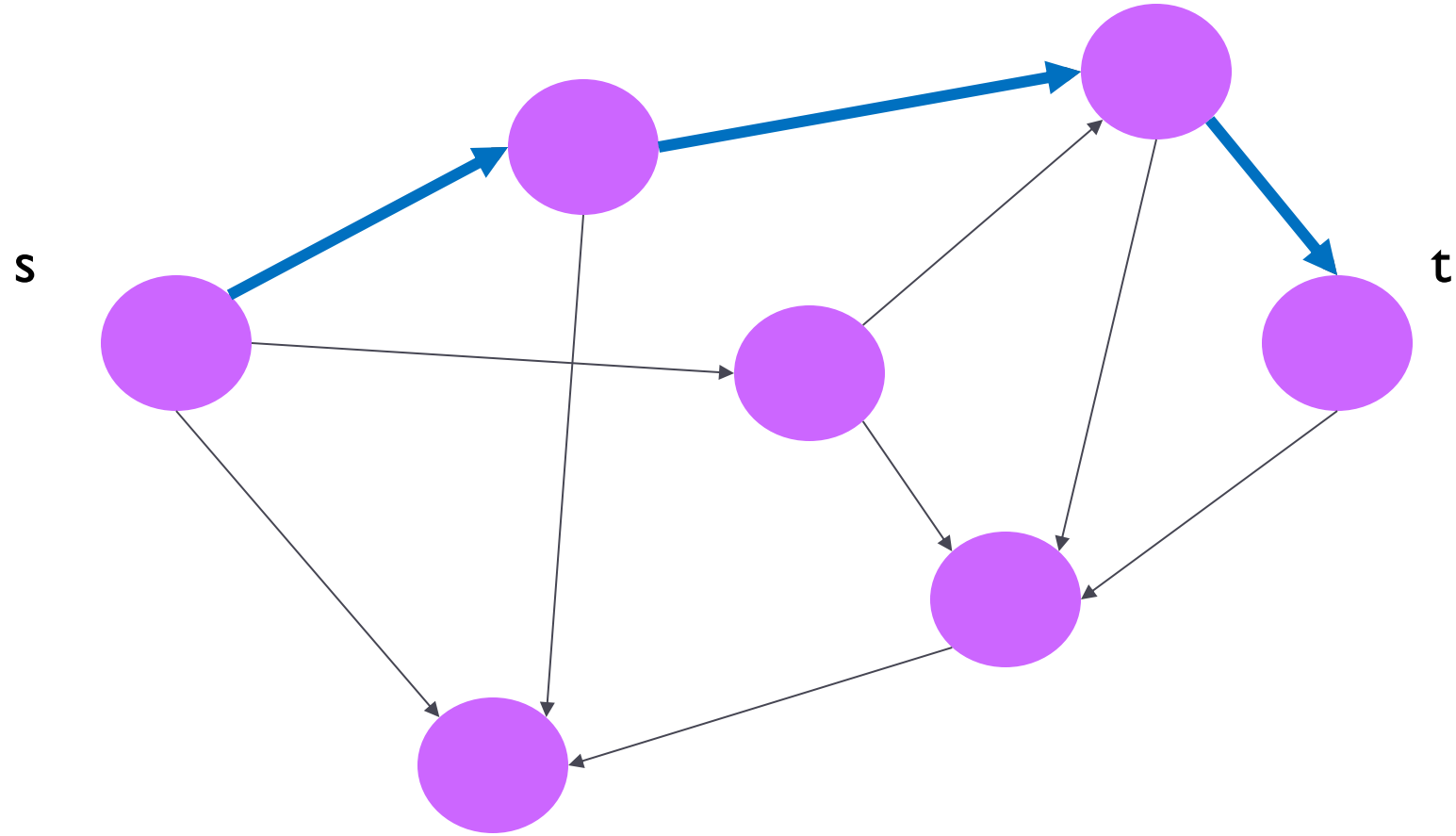
USTCON

- Undirected graph st-connectivity



Graph Connectivity

Is it in NL?





PATH is in NL

- Start at s
- For $i = 1, .., |V|$ {
  - Non-deterministically choose a neighbor and jump to it
  - Accept if you get to t

  }
- Reject

- Counting up to $|V|$ requires $\log|V|$ space
- Storing the current position requires $\log|V|$ space
- (Non-deterministic choice may take $\log|V|$ space)



Log-Space Reductions

Definition: A is log-space reducible to B, written $A \leq_L B$,

if there exists a log space TM M that, given input w, outputs f(w) s.t.

$w \in A$ iff $f(w) \in B$



NL-Completeness

Definition:

A language B is NL-Complete if

1. $B \in NL$
2. For every $A \in NL$, $A \leq_L B$

If $B \in L$, $A \leq_L B$, then $A \in L$?

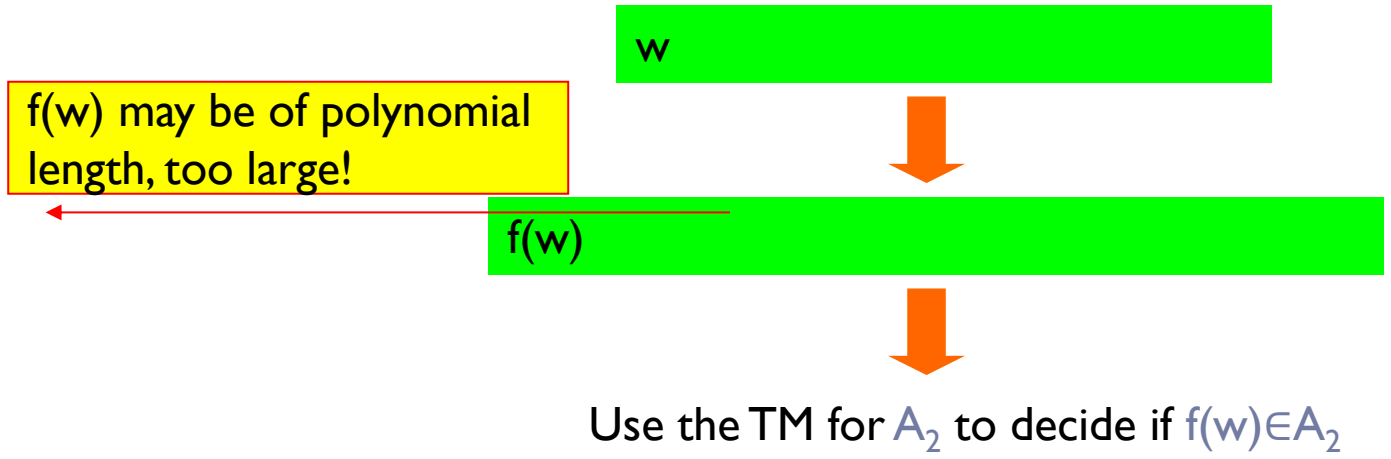


Do Log-Space Reductions imply what they should?

Suppose $A_1 \leq_L A_2$ and $A_2 \in L$

How to construct a log space TM which decides A_1?

Wrong Solution:





Log-Space reductions

Claim: if

1. $A_1 \leq_L A_2$ – f is the log-space reduction
2. $A_2 \in L$ – M is a log-space machine for A_2

Then, A_1 is in L

Proof: on input x, in or not-in A_1:

- Simulate M on f(x) and whenever M reads the ith symbol of its input tape
- Run f on x and wait for the ith bit to be outputted
  - i is bounded by poly(n), so it takes O(log n) bits



More generally

- If f and g are log-space computable functions, f∘g is also log-space computable

- g'(x,i), the i-th bit of g(x), is computable in log-space:
  - We just wait until the i-th bit comes out
  - Size of g(x) polynomial of |x|, so size of i is O(log|x|)
- f(g(x)):
  - Simulate the function f:
  - Whenever f reads the i-th bit on g(x), call g'(x,i)



Log-space Reduction

- Of course, log-space reduction is more restricted than polynomial-time reduction

- But it is also very powerful:
  - All NP-complete problems *known so far* can be reduced by log-space reductions

OPEN PROBLEM
Is NP-Completeness closed under log-space reduction?



NL-Completeness

Definition:

A language B is NL-Complete if

1. $B \in NL$
2. For every $A \in NL$, $A \leq_L B$

If (2) holds, B is NL-hard



Savitch's Theorem

Theorem:

$NL \subseteq SPACE(\log^2 n)$

or: $NL \subseteq L^2$

Proof:

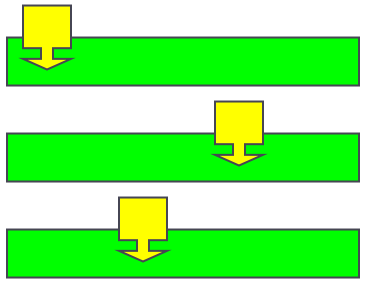
1. First prove PATH is NL-complete (under log-space reductions)
2. Then show an algorithm for PATH that uses $O(\log^2 n)$ space



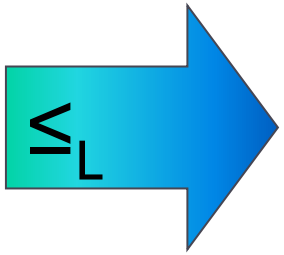
PATH is NL-Complete

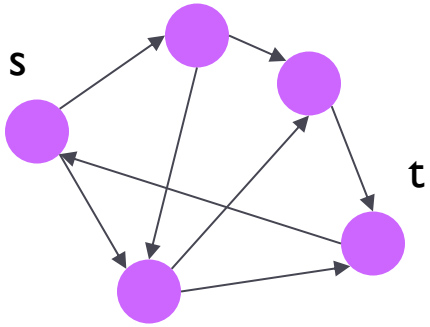
Theorem: PATH is NL-Complete

Proof: by the following reduction:



"Does M accept x?"





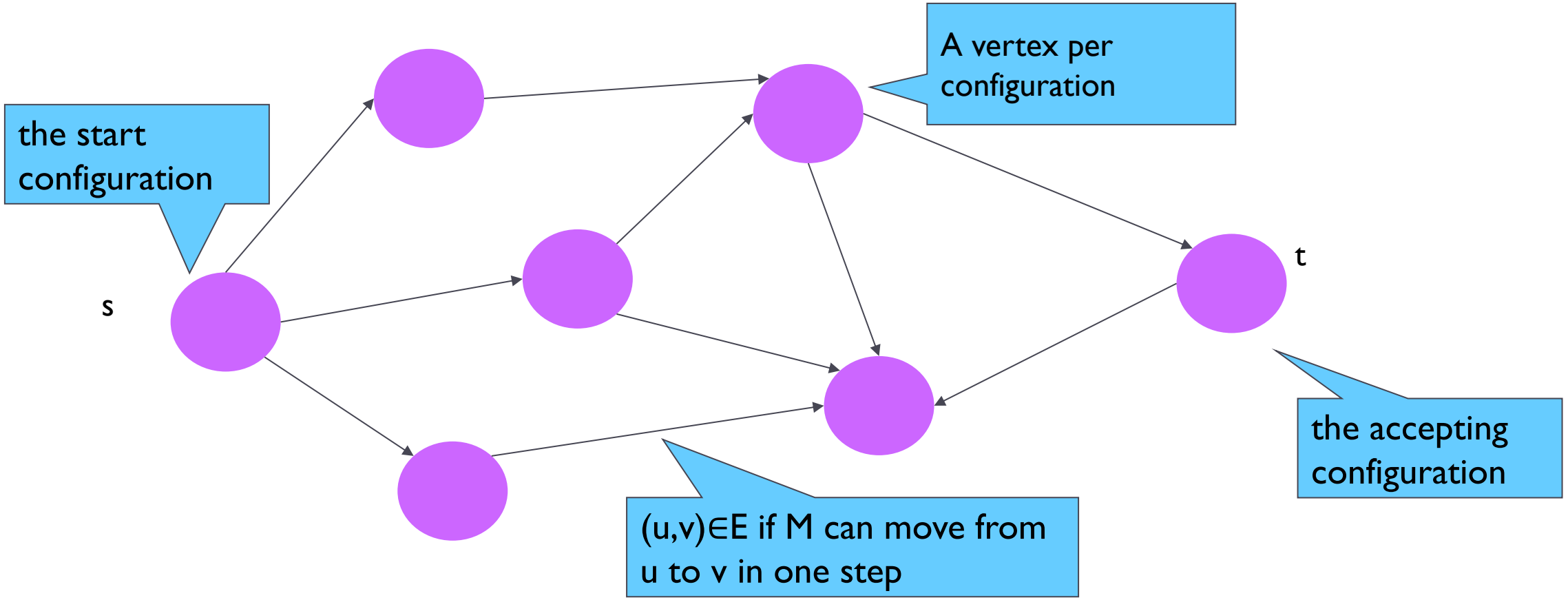
"Is there a path from s to t?"



Configurations Graph

(Without loss of generality, we can assume all NTM's have exactly one accepting configuration)

A Computation of a NTM M on an input x can be described by a graph $G_{M,x}$:



Correctness

Claim: For every non-deterministic log-space Turing machine M and every input x, M accepts x iff there is a path from s to t in $G_{M,x}$



PATH is NL-Complete

Corollary: PATH is NL-Complete

Proof: We've shown PATH is in NL. We've also presented a reduction from any NL language to PATH which is computable in log-space (Why?)

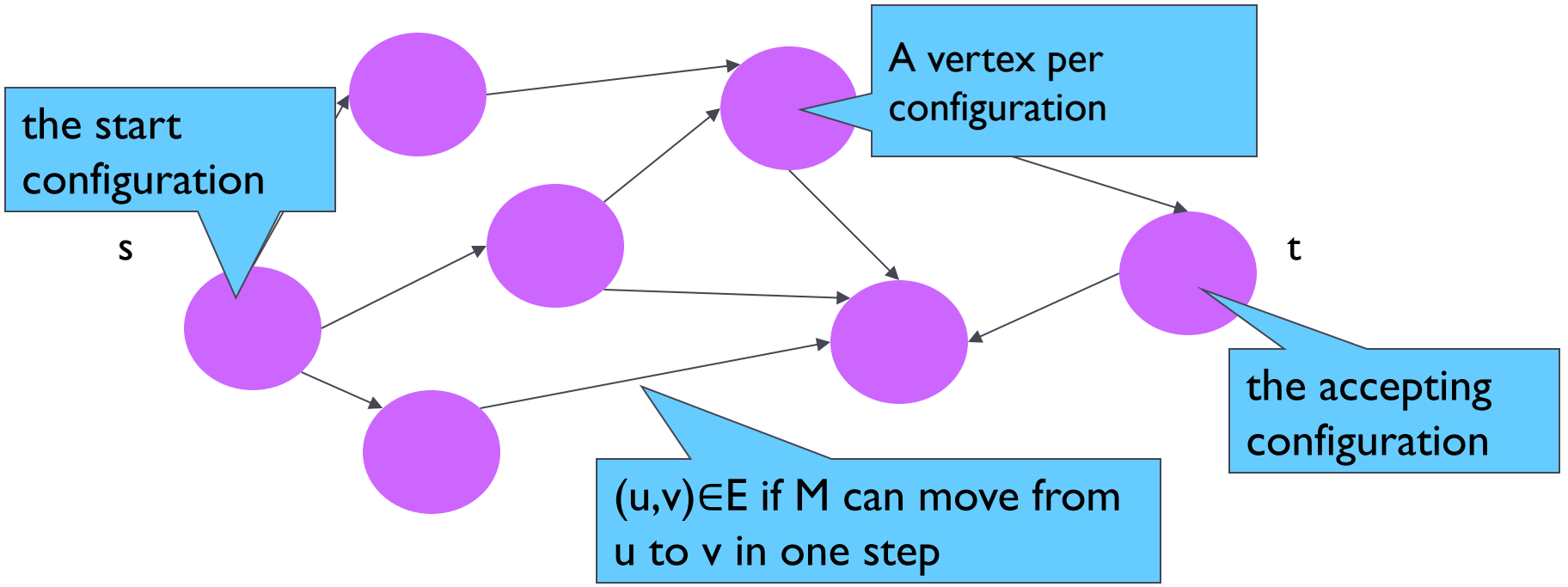
- Suppose we have a nondeterministic log-space TM M and input x, and n=|x|

Claim: The number of possible configurations is $2^{O(\log n)}$, which is polynomial of n



PATH is NL-Complete

- The graph is log-space computable from M, x
  - Enumerating all nodes takes O(log n) space
  - For every node, check all edges starting from it





PATH is NL-Complete

- The graph is log-space computable from M, x
  - Enumerating all nodes takes O(log n) space
  - For every node, check all edges starting from it

- So we have a log-space reduction from M, x to a PATH instance



A Byproduct

Claim: $NL \subseteq P$

Proof:

- Any NL language is log-space reducible to PATH
- Thus, any NL language is poly-time reducible to PATH
- PATH is in P
- Thus any NL language is in P ■



What Next?

Theorem:

$$NSPACE(\log n) \subseteq SPACE(\log^2 n)$$

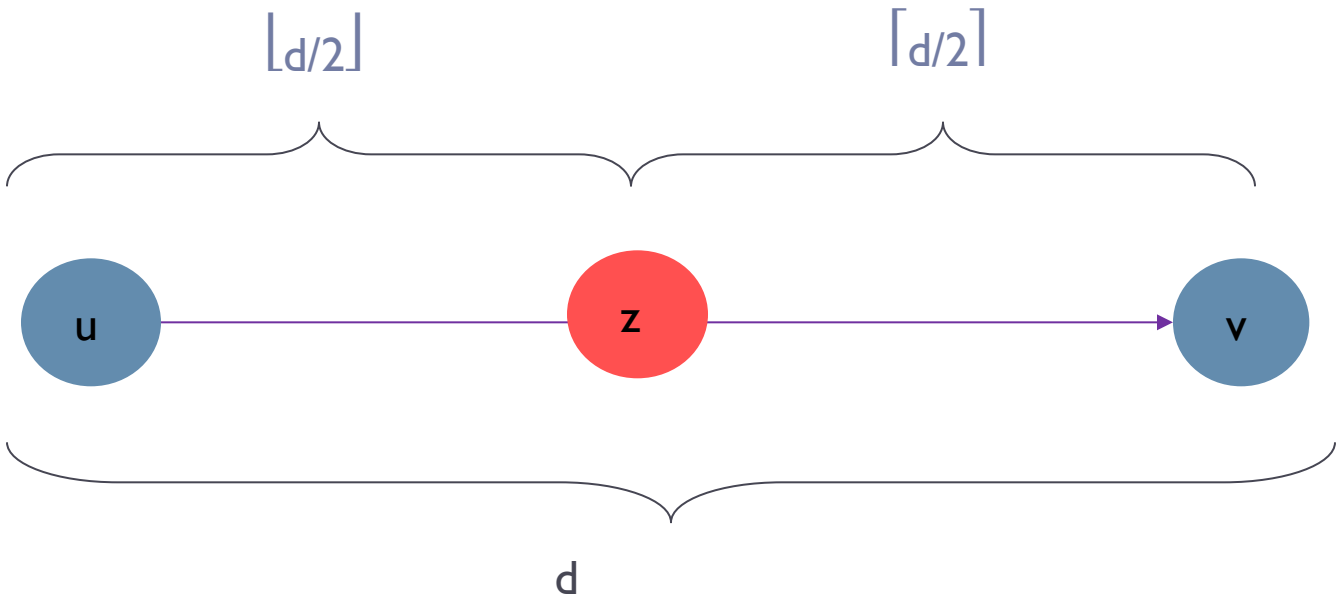
- We have shown PATH is NL-Complete
- We need to show PATH can be decided by a deterministic TM in $O(\log^2 n)$ space



The Trick

"Is there a path from u to v of length ≤d?"

"Is there a vertex z, so there is a path from u to z of length ≤⌊d/2⌋ and one from z to v of length ≤⌈d/2⌉?"





The Algorithm

```
Boolean PATH(a,b,d) {      //whether there is path from a to b with distance ≤d
        if there is an edge from a to b then
                return TRUE
        else {
                if d=1 return FALSE
                for every vertex v {
                        if PATH(a,v, ⌊d/2⌋) and PATH(v,b, ⌈d/2⌉)
                        then return TRUE
                }
                return FALSE
        }
}
```



The Algorithm

```
Boolean PATH(a,b,d) {      //whether there is path from a to b with distance ≤d
        if there is an edge from a to b then
                return TRUE
        else {
                if d=1 return FALSE
                for every vertex v {
                        if PATH(a,v, ⌊d/2⌋) and PATH(v,b, ⌈d/2⌉)
                        then return TRUE
                }
                return FALSE
        }
}
```

In the recursion stack:

- Each call takes O(log n) space (to keep track of v and d)
- *log n* recursion depth



$O(\log^2 n)$ Space DTM

Claim: There is a deterministic TM which decides PATH in $O(\log^2 n)$ space

Proof:

To solve PATH, we invoke PATH(s,t,|V|)

The space complexity:

$S(n,d)=S(n,d/2)+O(\log n)=O(\log n \cdot \log d)$

$S(n,n-1)=O(\log^2 n)$



Graph Connectivity

PATH (st-connectivity)

- Instance: a directed graph $G=(V,E)$ and two vertices $s,t \in V$
- Problem: To decide if there is a path from s to t in G?
- it is in NL, and in L^2 ($SPACE(\log^2 n)$)

USTCON

- Undirected graph st-connectivity
- $\in$RL (randomized logspace) by simple random walk
- $\in$L
  - *Reingold, Omer. Undirected ST-connectivity in log-space. STOC'05*



Conclusion

Theorem:

$NSPACE(\log n) \subseteq SPACE(\log^2 n)$

How about the general case
$NSPACE(S(n)) \subseteq SPACE(S^2(n))$?



Savitch's Theorem—Generalized form

Theorem:

$\forall S(n) \geq \log(n)$

$NSPACE(S(n)) \subseteq SPACE(S(n)^2)$

Proof:

we'll use $NL \subseteq SPACE(\log^2 n)$ to show the general case



Padding argument-- Example

- Claim. If P=NP, then EXP=NEXP
- Proof. We need to show NEXP $\subseteq$ EXP



Padding argument-- Example

- Claim. If P=NP, then EXP=NEXP
- Proof. We need to show NEXP ⊆ EXP

- Suppose a language L∈NEXP, then some NTM M decides L in 2^{n^c} time

- Let

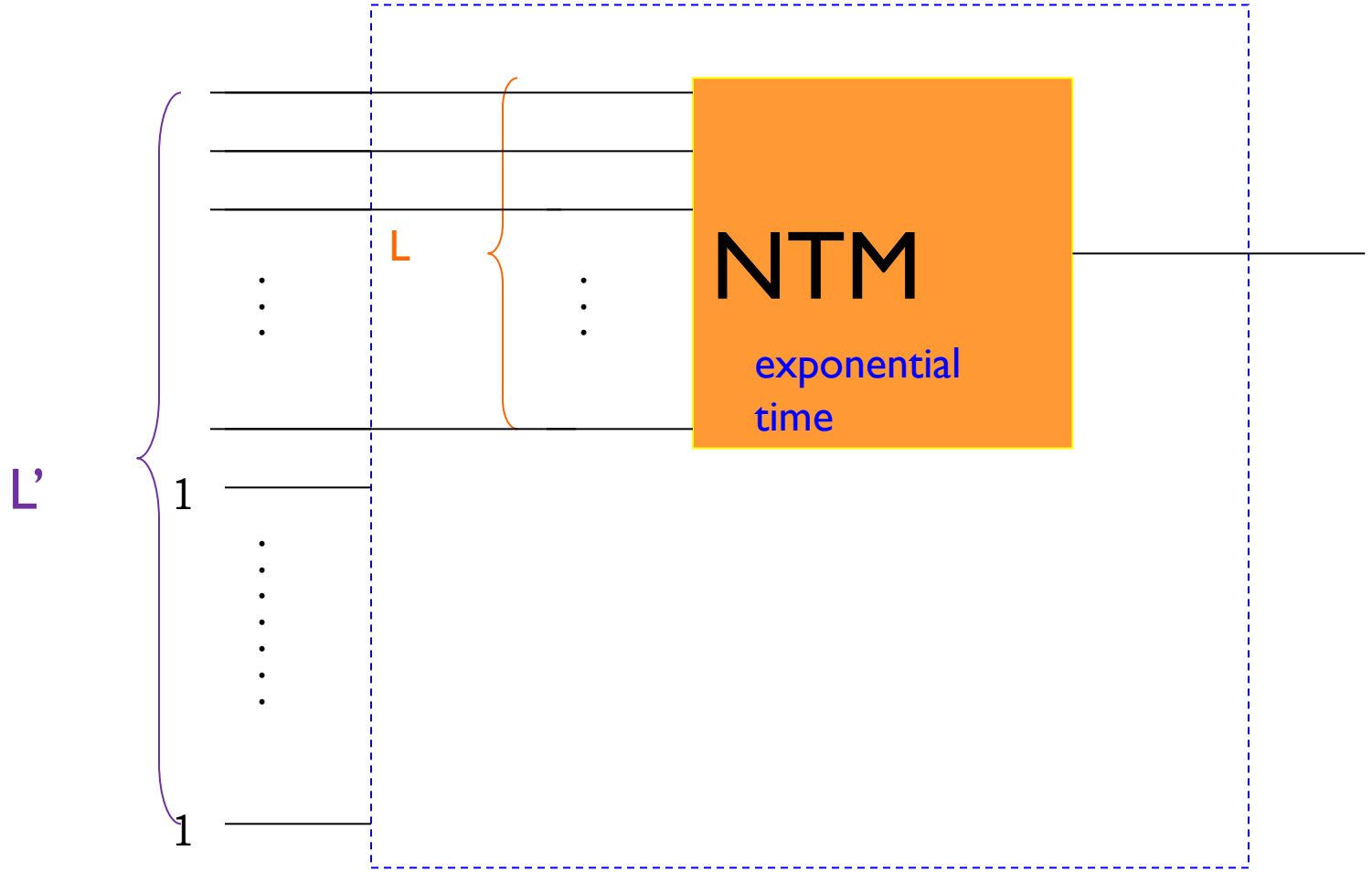
$$L' = \{x1^{2^{|x|^c}} \mid x \in L\}$$

  - where 1 is a symbol not in L



Idea

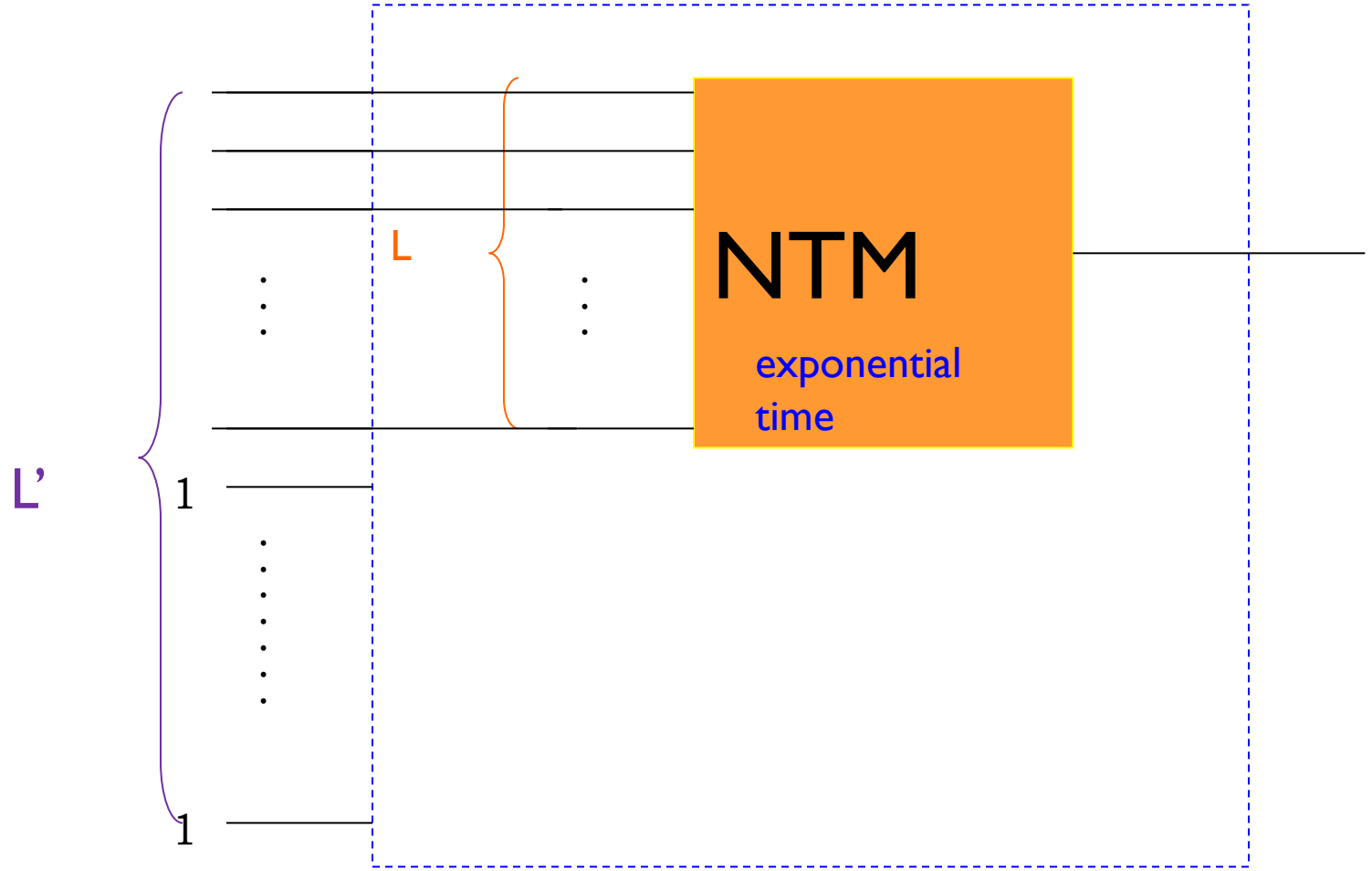
Claim. If P=NP, then EXP=NEXP



$$L' = \{x1^{2^{|x|^c}} \mid x \in L\}$$

Idea

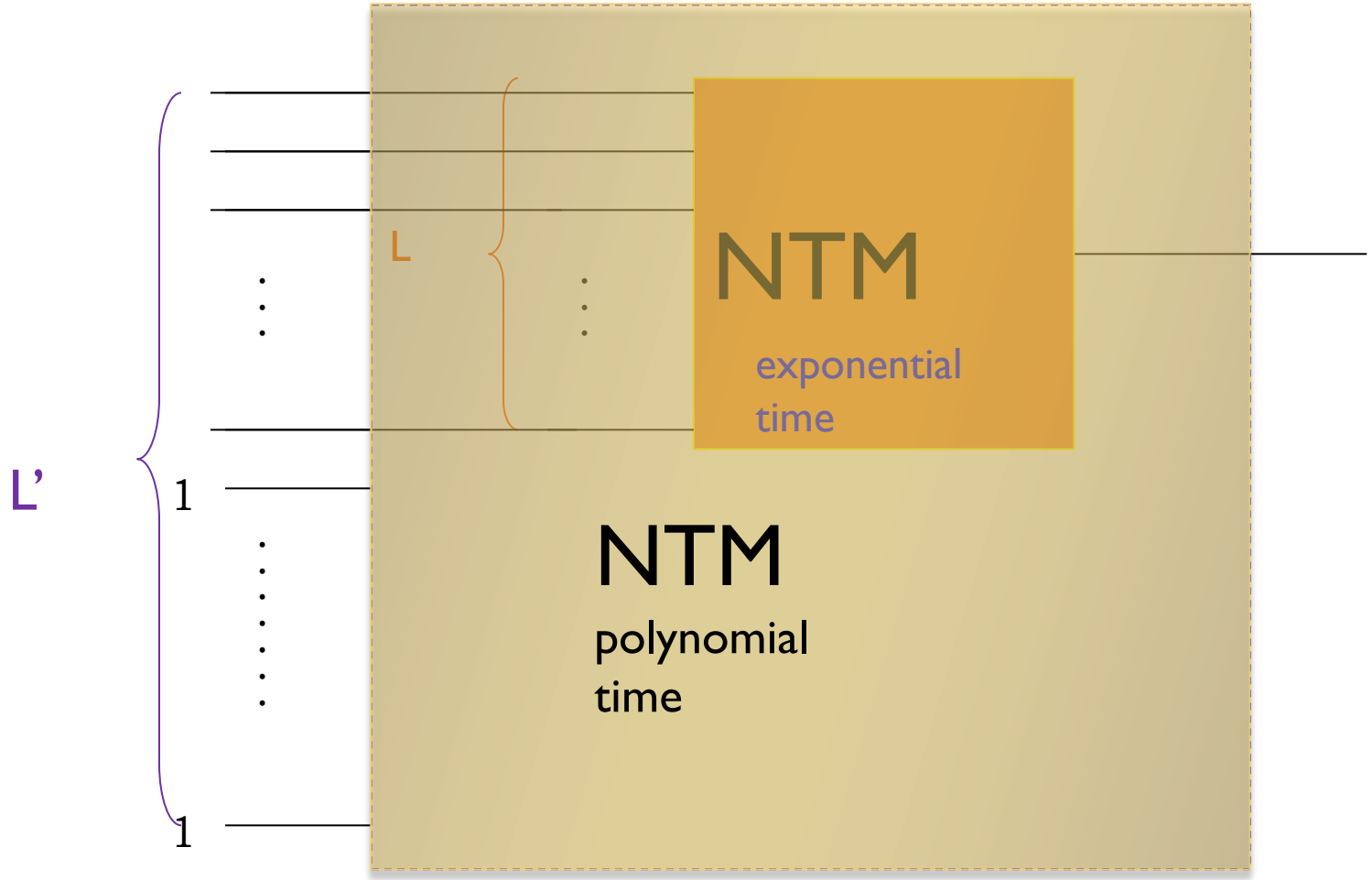
Claim. If P=NP, then EXP=NEXP



L' can be decides by a polynomial time NTM

Idea

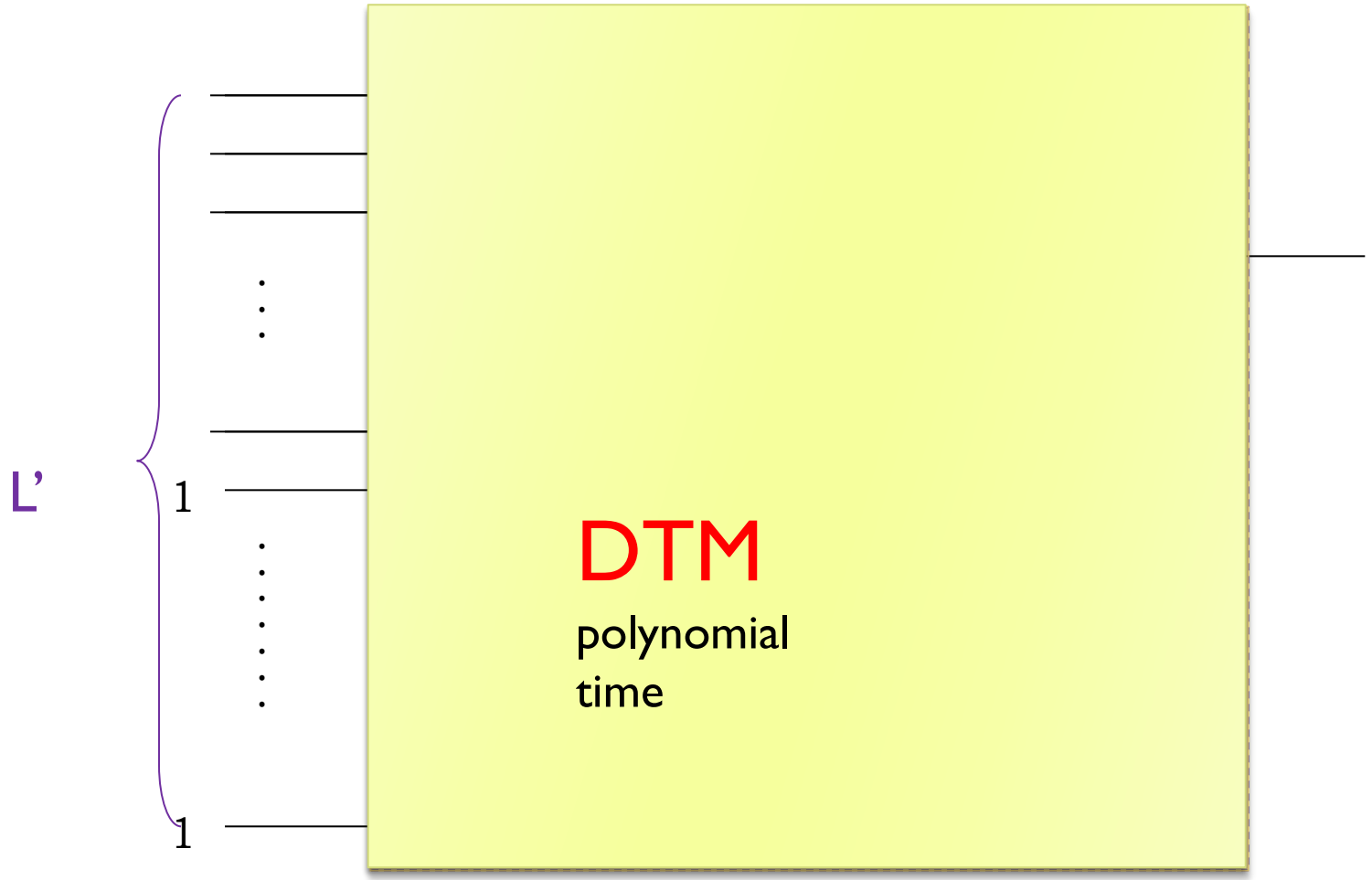
Claim. If P=NP, then EXP=NEXP



L' can be decides by a polynomial time NTM

Idea

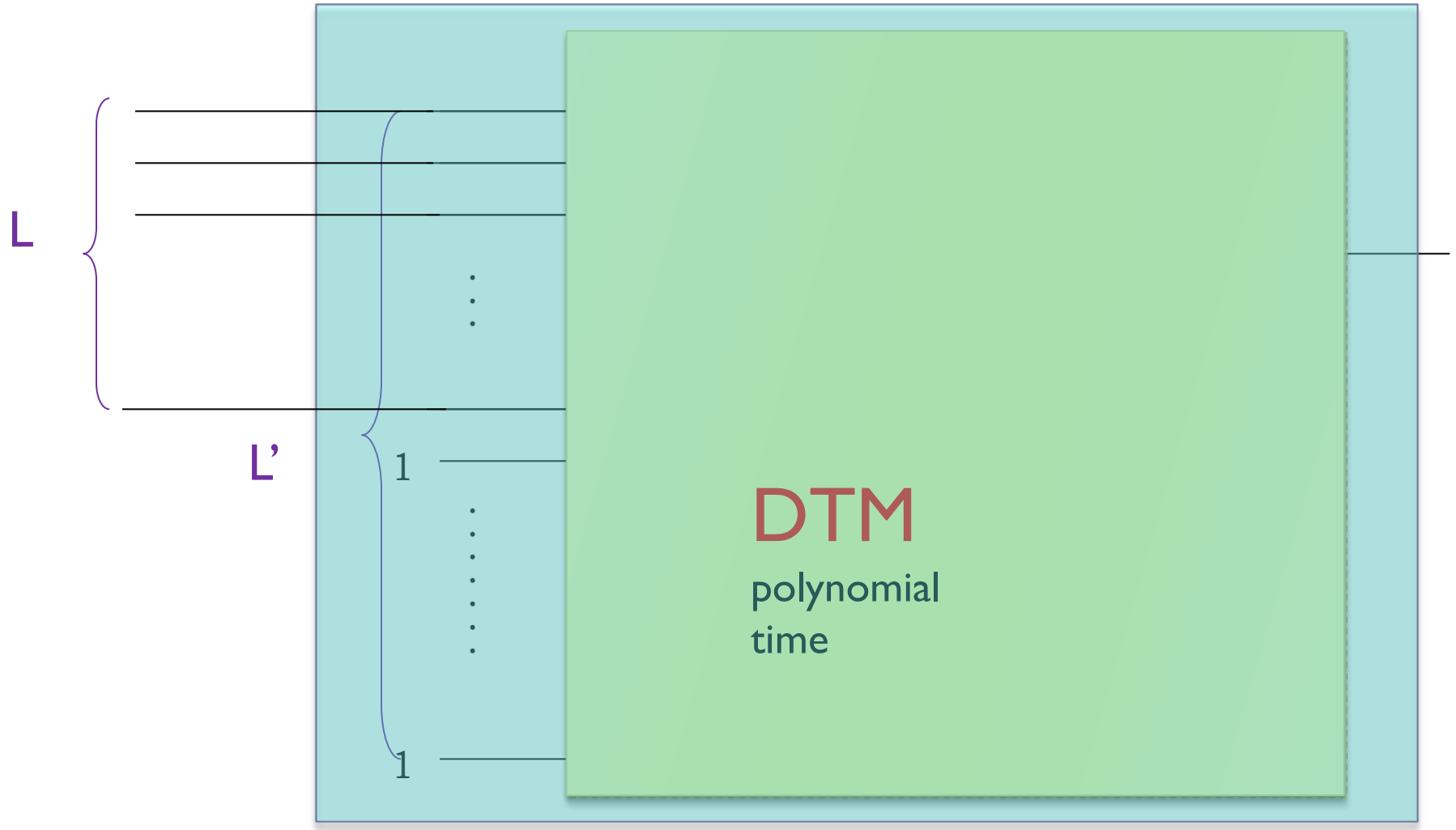
Claim. If P=NP, then EXP=NEXP



By assumption P=NP, L' ∈ P

Idea

$$L' = \{x1^{2^{|x|^c}} \mid x \in L\}$$



By assumption P=NP, L' ∈ P, so L∈ EXP

Padding argument

- Claim. If P=NP, then EXP=NEXP
- Proof. We need to show NEXP ⊆ EXP

- Suppose a language L∈NEXP, then some NTM M decides L in 2^{n^c} time

- Let $L' = \{x1^{2^{|x|^c}} \mid x \in L\}$
  - L' ∈ NP, so L' ∈ P, there is a polynomial time DTM M' for L'
  - To check whether x∈L, run $M'(x1^{2^{|x|^c}})$

- So L∈EXP



Time/Space constructable functions

Definition

A function $f: \mathbb{N} \rightarrow \mathbb{N}$, where $f(n) \geq \log n$, is called space constructible, if the function that maps 1^n to the binary representation of $f(n)$ is computable in space $O(f(n))$.

Definition

A function $f: \mathbb{N} \rightarrow \mathbb{N}$, where $f(n) \geq n \log n$, is called time constructible, if the function that maps 1^n to the binary representation of $f(n)$ is computable in time $O(f(n))$.

Examples

- polynomials
- $n \log n$
- 2^n, 3^n,…

All commonly used functions are space/time constructible



Savitch's Theorem—Generalized form

Theorem:

For any space constructible function $S(n) \geq \log n$,

$$NSPACE(S(n)) \subseteq SPACE(S(n)^2)$$

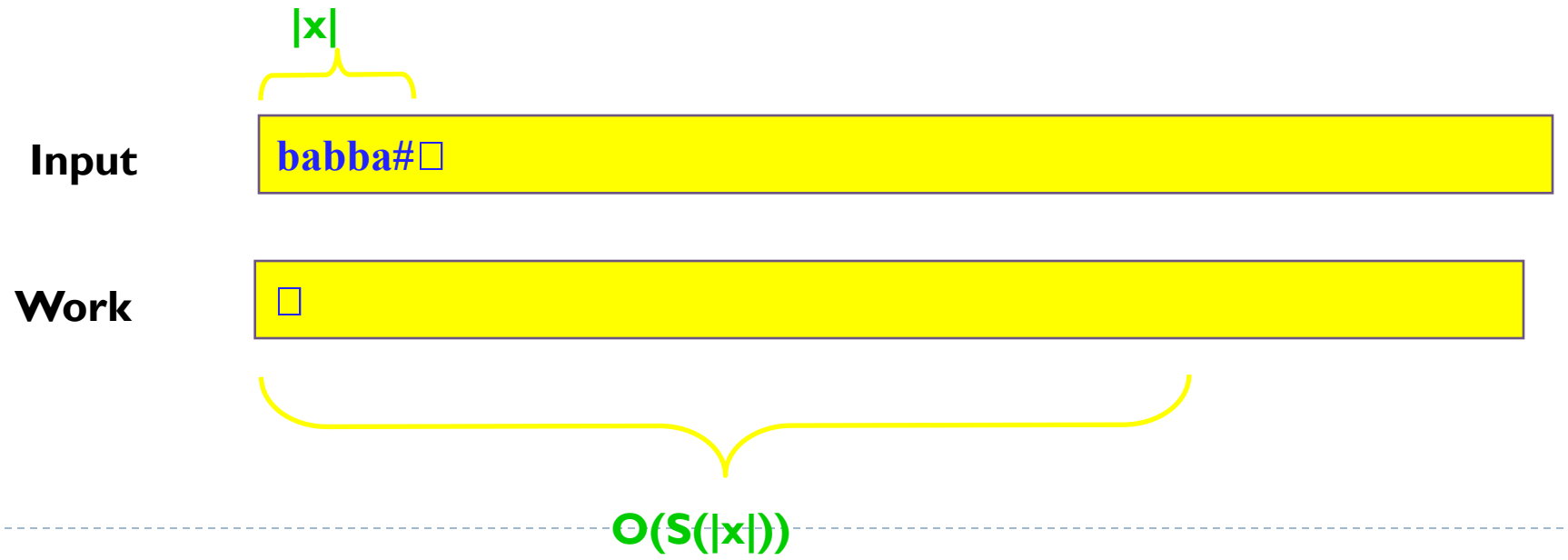
Proof:

by padding argument on $NL \subseteq SPACE(\log^2 n)$



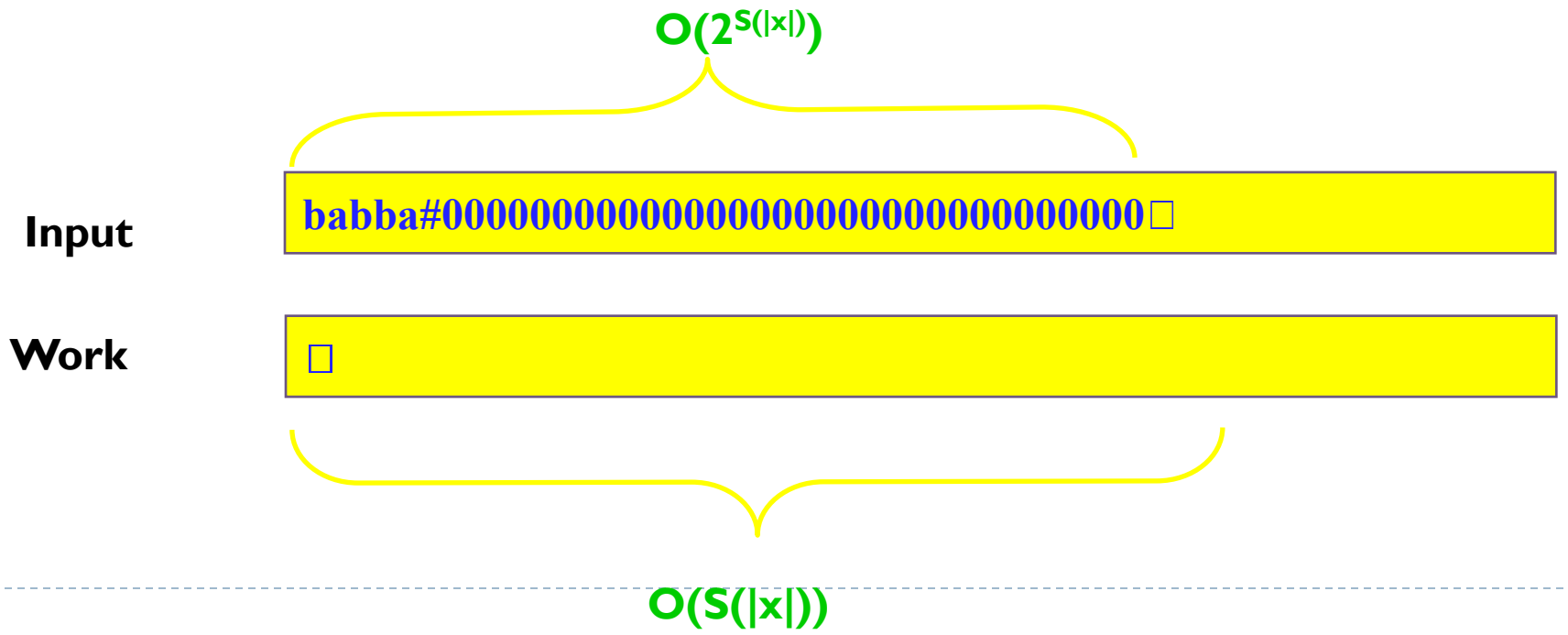
Padding argument

- Let $L \in NSPACE(S(n))$
- There is a 3-Tape NTM M_L:



Padding argument

- Let L∈NSPACE(S(n))
- Let $L' = \{ x\#0^{2^{S(|x|)}} \mid x \in L \}$
- We'll show a NTM $M_{L'}$ which decides L' in the same number of cells as M_L



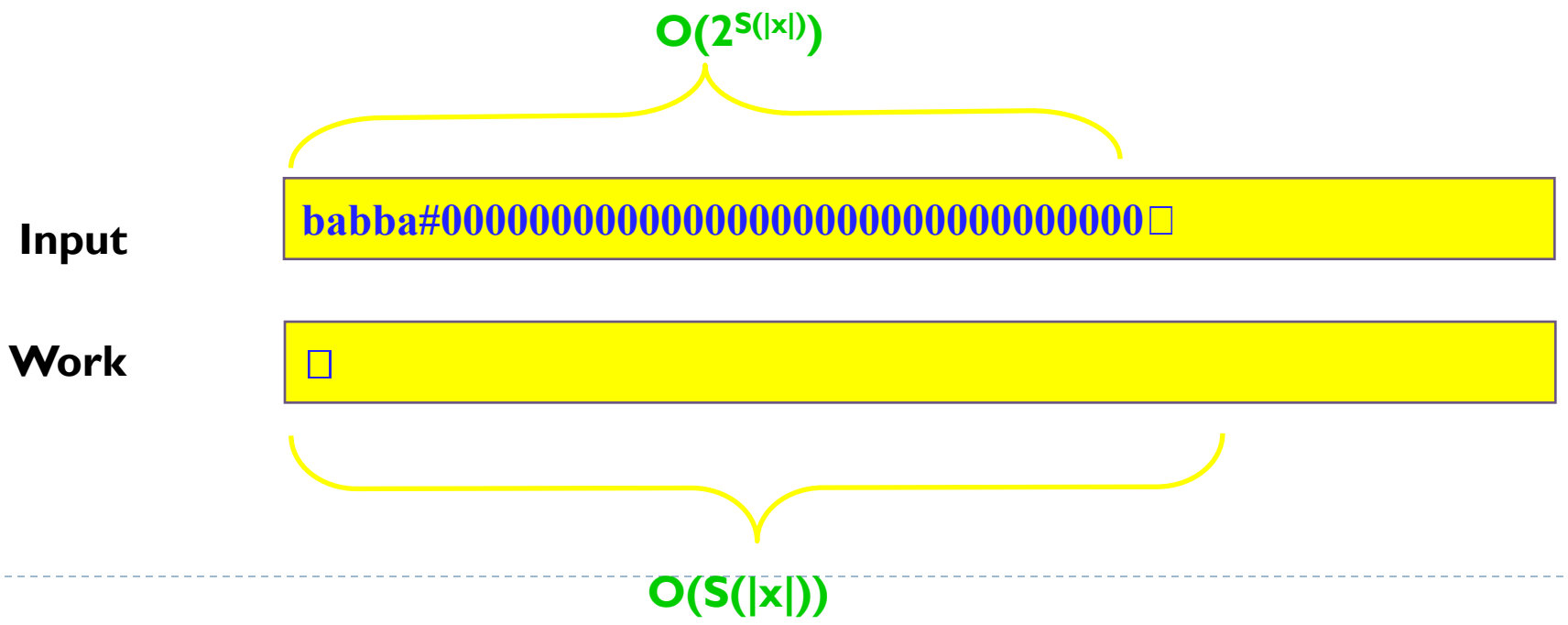
Padding argument – M_L'

In O(S(|x|)) space

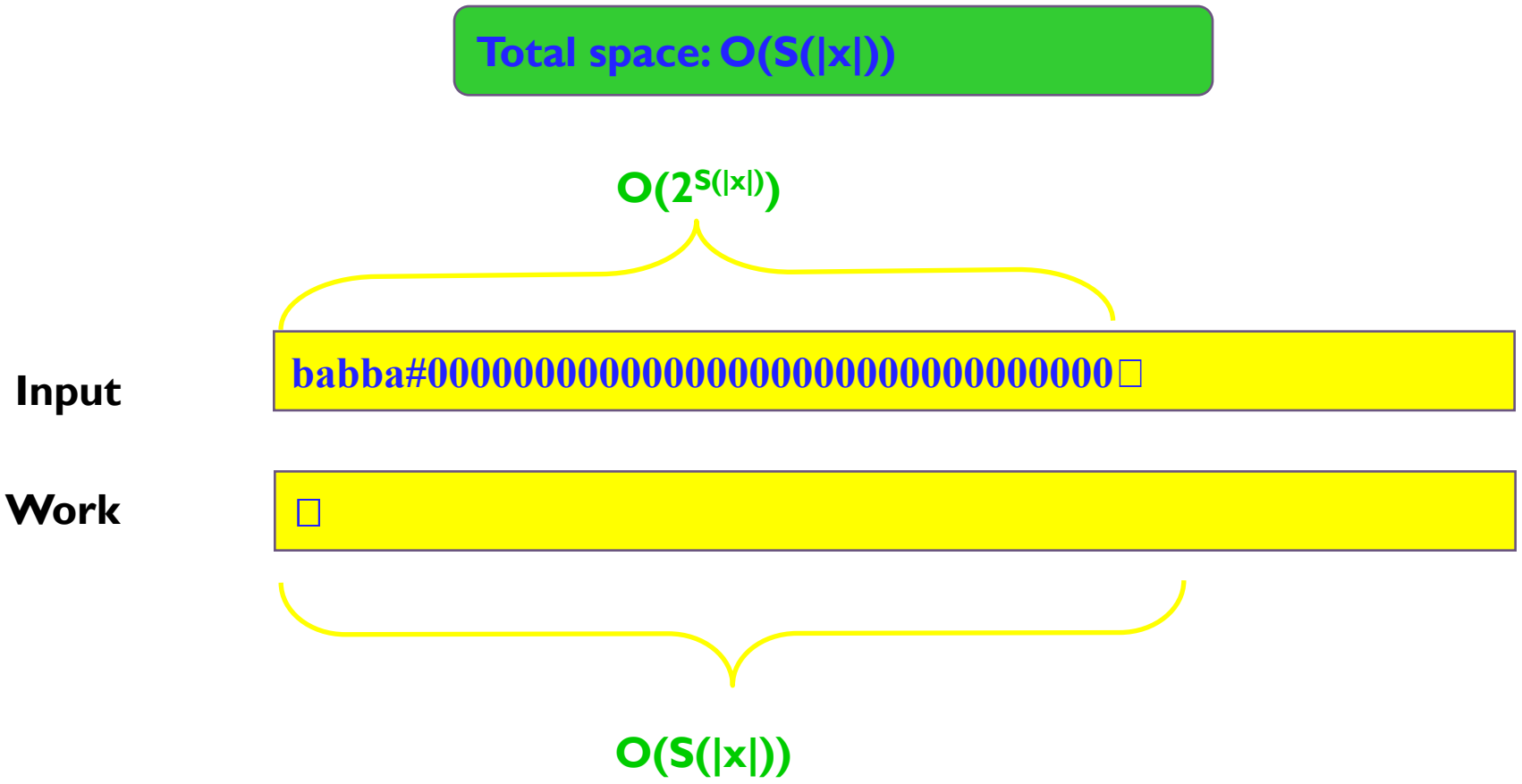
Note that S(n) can be computed in O(S(n)) space

1. Count backwards the number of 0's and check there are $2^{S(|x|)}$ such
2. Run M_L on x

in O(S(|x|)) space



Padding argument



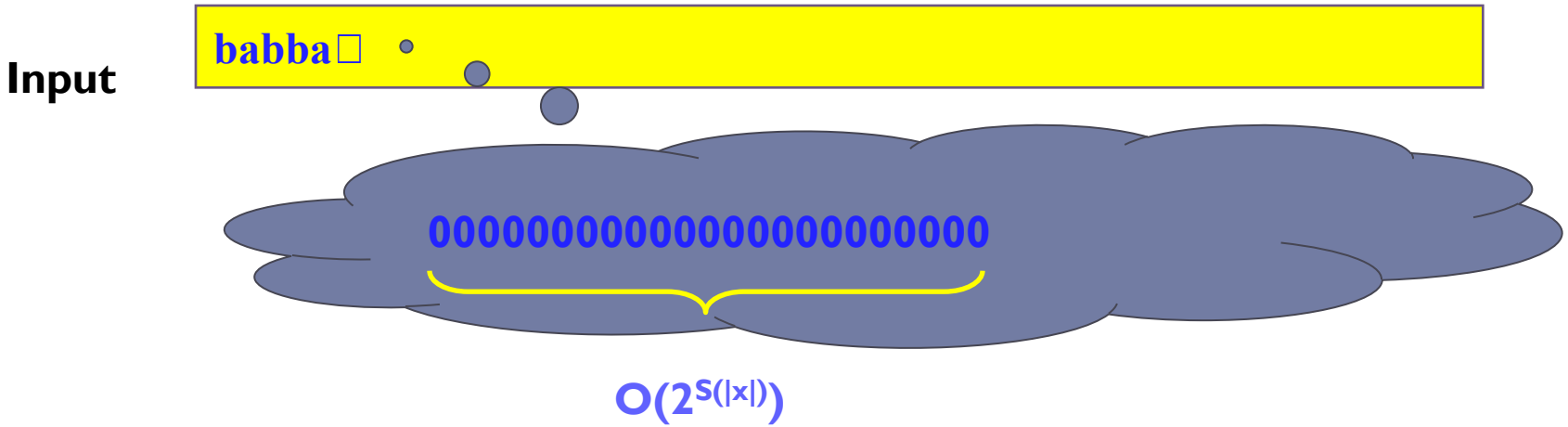
Padding argument

- We started with $L \in NSPACE(S(n))$
- We showed: $L' \in NL$
- Thus, $L' \in SPACE(\log^2 n)$
- Using the DTM for L' we'll construct a DTM for L, which will work in $O(S(n)^2)$ space



Padding argument

- The DTM for L will simulate the DTM for L' when working on its input concatenated with zeros





Padding argument

- We started with $L \in NSPACE(S(n))$
- We showed: $L' \in NL$
- Thus, $L' \in SPACE(\log^2 n)$
- Using the DTM for L' we'll construct a DTM for L, which will work in $O(S(n)^2)$ space

- Thus our machine uses $O(S(n)^2)$ space
  - Maintaining the simulated position (on the imaginary part of the tape) takes $\log 2^{O(S(|x|))} = O(S(|x|))$ space
- $\Rightarrow NSPACE(S(n)) \subseteq SPACE(S(n)^2)$

- What about PSPACE and NPSPACE?



Corollary

Corollary: PSPACE = NPSPACE

Proof:

Clearly, PSPACE⊆NPSPACE

By Savitch's theorem, NPSPACE⊆PSPACE ■



Space Vs. Time

- We've seen space complexity probably doesn't resemble time complexity:
  - Non-determinism doesn't decrease the space complexity drastically (Savitch's theorem)
  - we don't know whether P=NP
- We'll next see another difference:
  - Non-deterministic space complexity classes are closed under completion (Immerman's theorem)
  - we don't know whether NP=co-NP



NON-PATH

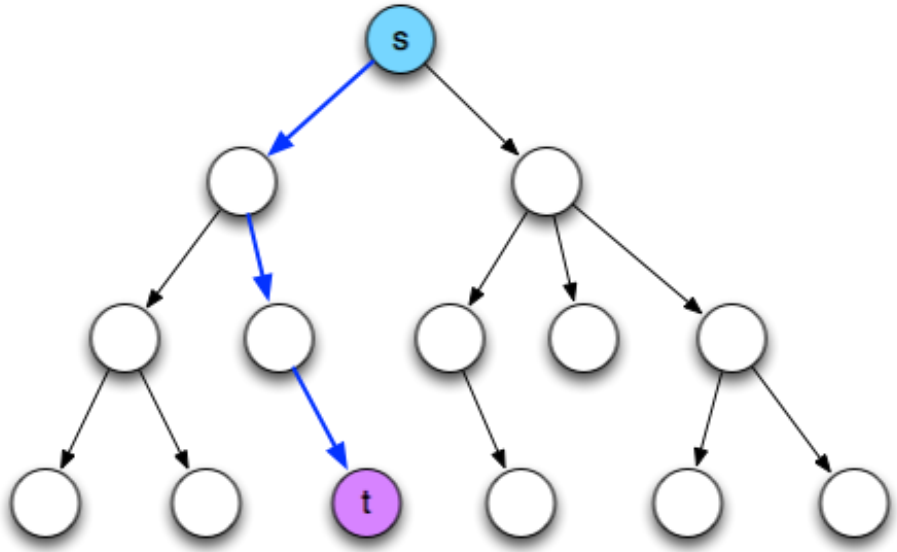
NON-PATH

- Instance: A directed graph G and two vertices $s,t \in V$
- Problem: To decide if there is no path from s to t



NON-PATH

- Clearly, NON-PATH is coNL-Complete
- If we can show it is also in NL, then NL=coNL
  - One of the branches accepts iff s cannot reach t
  - Much harder than finding a path





Immerman-Szelepcsenyi Theorem

Theorem *[Immerman/Szelepcsenyi, 1987]:* NL=coNL

Proof:

(1) NON-PATH is coNL-Complete

(2) NON-PATH$\in$NL

Hence, NL=coNL ■



I-S Theorem

We'll see a log-space nondeterministic algorithm for counting reachability

1. Count how many vertices are reachable from s
2. Take out t and count again
3. Accept if the two numbers are the same



I-S Theorem

1. Count how many vertices are reachable from s
2. Take out t and count again
3. Accept if the two numbers are the same

Failed trial: (what's wrong?)

```
counter=1;
For each node v≠s
        Nondeterministically follow a path from s and reject if doesn't end in v;
        counter++;
```

For a vertex v not reachable from s, all branches will reject



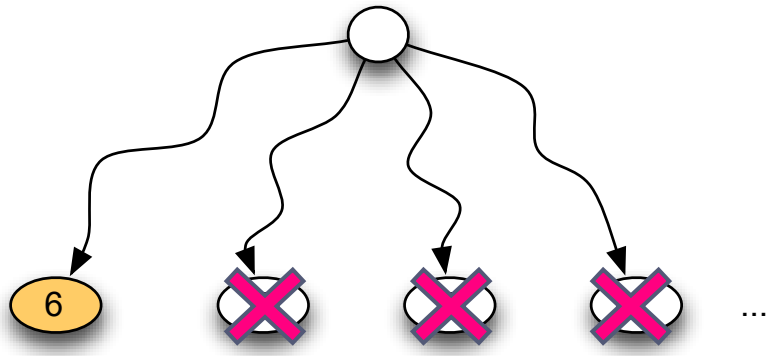
```
counter=1;
For each node v≠s
        Nondeterministically perform or skip: (guess whether v is reachable)
                Nondeterministically follow a path from s
                        and reject if doesn't end in v;
                counter++;
```

	Reachable	Not reachable
Guess reachable	✅	Reject ✅
Guess not reachable	Not reject and underestimate	✅



```
counter=1;
For each node v≠s
        Nondeterministically perform or skip: (guess whether v is reachable)
                Nondeterministically follow a path from s
                        and reject if doesn't end in v;
                counter++;
```

	Reachable	Not reachable
Guess reachable	✅	Reject ✅
Guess not reachable	Not reject and underestimate	✅



We need to reject on

```
counter=1;
For each node v≠s
        Guess whether v is reachable
        If yes,
                Nondeterministically follow a path from s
                        and reject if doesn't end in v;
                counter++;
        If no,
                if v is reachable from s, we need to reject
```



Number of nodes reachable from s

- R(i) = # nodes reachable from s in at most i steps
- R(0) = 1: node s

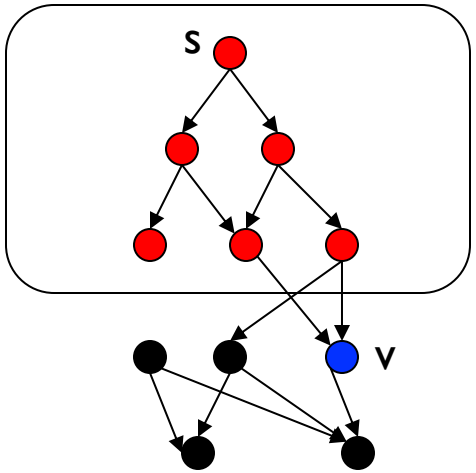
- we will compute R(i+1) from R(i) using O(log n) space and nondeterminism

- computation paths with "bad guesses" all lead to reject



Number of nodes reachable from s

- computing R(i+1) from R(i):



R(2) = 6

- for each v ∈V, *guess* if v reachable from s in at most i+1 steps
- if we guess no, then we count R(i) again without vertices pointing to v
- if R(i) changed, we guess wrong



computing R(i+1) from R(i)

```
counter=1;
For each node v≠s
        Guess whether v is reachable within i+1 steps
        If yes,
                Nondeterministically follow a path from s within i+1 steps
                        and reject if doesn't end in v;
                counter++;
        Else,
                counter2=1;
                For each node u≠s
                        Guess if u is reachable within i steps
                                Nondeterministically follow a path from s
                                within i steps and reject if doesn't end in u;
                                if u is v or there is an edge (u,v), reject
                                counter2++;
                If counter2 ≠ R(i), reject
R(i+1)=counter;
```


When guessing v is not reachable, if v is reachable, we must reject!

computing R(i+1) from R(i)

```
counter=1;
For each node v≠s
        Guess whether v is reachable within i+1 steps
        If yes,
                Nondeterministically follow a path from s within i+1 steps
                        and reject if doesn't end in v;
                counter++;
        Else,
                counter2=1;
                For each node u≠s
                        Guess if u is reachable within i steps
                                stically follow a path from s
                                and reject if doesn't end in u;
                                e is an edge (u,v), reject
                        counter2++;
                If counter2 ≠ R(i), reject
R(i+1)=counter;
```

Since we already know R(i), we can reject on all the branches guessing wrong (underestimated results)



computing R(i+1) from R(i)

counter=1;

The only branch that can go through these steps without rejecting is:
1. All guesses are correct
2. there is no u reachable from s within i steps and connected to v within 1 step

So v is not reachable from s within i+1 steps

When v is reachable from s within i+1 steps, the "no" guesses will all reject

Else,

counter2=1;

For each node u≠s

Guess if u is reachable within i steps

Nondeterministically follow a path from s within i steps and reject if doesn't end in u;

if u is v or there is an edge (u,v), reject

counter2++;

If counter2 ≠ R(i), reject

R(i+1)=counter;



NON-PATH∈NL

- Compute R(1),R(2),…,R(n)
  - R(n) is the number of vertices reachable from s
- Then we delete t and compute R'(n) again
- If R(n)=R'(n), accept, otherwise reject

Hence, NL=coNL



Corollary

Corollary:

$\forall s(n) \geq \log(n)$, $NSPACE(s(n)) = coNSPACE(s(n))$

Proof: By a padding argument



PSPACE

- Class of languages that are decidable by a DTM in polynomial space

- e.g. SAT
  - $(x_1 \vee x_2) \wedge (\sim x_2 \vee x_3 \vee \sim x_4) \wedge (\sim x_1)$
  - For each variable, assign true or false, check satisfiability
  - Takes linear space

- Since SAT is NP-complete, so NP $\subseteq$ PSPACE
  - unlikely they are equal



TQBF (true quantified Boolean formula)

- We can use the insight of Savitch's proof to show a language which is complete for PSPACE
- We present TQBF, which is the quantified version of SAT



- If x_i does not occur free in $\mathcal{A}$, then
  - $\vdash_K (\forall x_i)(\mathcal{A} \rightarrow \mathcal{B}) \leftrightarrow (\mathcal{A} \rightarrow (\forall x_i)\mathcal{B})$
  - $\vdash_K (\exists x_i)(\mathcal{A} \rightarrow \mathcal{B}) \leftrightarrow (\mathcal{A} \rightarrow (\exists x_i)\mathcal{B})$

- If x_i does not occur free in $\mathcal{B}$, then
  - $\vdash_K (\forall x_i)(\mathcal{A} \rightarrow \mathcal{B}) \leftrightarrow ((\exists x_i)\mathcal{A} \rightarrow \mathcal{B})$
  - $\vdash_K (\exists x_i)(\mathcal{A} \rightarrow \mathcal{B}) \leftrightarrow ((\forall x_i)\mathcal{A} \rightarrow \mathcal{B})$

How do you comprehend this?



Prenex form

- The form $(Q_1 x_{i1})(Q_2 x_{i2})\ldots(Q_k x_{ik})\mathcal{D}$, where $\mathcal{D}$ is a wf. of $\mathcal{L}$ with no quantifiers, and each Q_j is either $\forall$ or $\exists$

- Theorem. Any wf. $\mathcal{A}$ is equivalent to a wf. $\mathcal{B}$ in prenex form
  - by:

If x_i occur free in $\mathcal{A}(x_i)$ and x_j is a variable which does not occur in $\mathcal{A}(x_i)$, then

$\vdash_K (\forall x_i)\mathcal{A}\,(x_i) \leftrightarrow (\forall x_j)\mathcal{A}\,(x_j)$

If x_i does not occur free in $\mathcal{A}$, then

$\vdash_K (\forall x_i)(\mathcal{A} \rightarrow \mathcal{B}) \leftrightarrow (\mathcal{A} \rightarrow (\forall x_i)\mathcal{B})$

$\vdash_K (\exists x_i)(\mathcal{A} \rightarrow \mathcal{B}) \leftrightarrow (\mathcal{A} \rightarrow (\exists x_i)\mathcal{B})$

If x_i does not occur free in $\mathcal{B}$, then

$\vdash_K (\forall x_i)(\mathcal{A} \rightarrow \mathcal{B}) \leftrightarrow ((\exists x_i)\mathcal{A} \rightarrow \mathcal{B})$

$\vdash_K (\exists x_i)(\mathcal{A} \rightarrow \mathcal{B}) \leftrightarrow ((\forall x_i)\mathcal{A} \rightarrow \mathcal{B})$



Prenex form

- The form $(Q_1 x_{i1})(Q_2 x_{i2})\ldots(Q_k x_{ik})\mathcal{D}$, where $\mathcal{D}$ is a wf. of $\mathcal{L}$ with no quantifiers, and each Q_j is either $\forall$ or $\exists$

- Theorem. Any wf. $\mathcal{A}$ is equivalent to a wf. $\mathcal{B}$ in prenex form

- Example:
  - change $((\forall x_1)\, A_1^2(x_1,x_2) \rightarrow \sim(\exists x_2)\, A_1^1(x_2)) \rightarrow (\forall x_1)(\forall x_2)\, A_1^2(x_1,x_2)$ to prenex form
  - first change the bound variables so that they are different from one another and from free variables:
  - $((\forall x_1)\, A_1^2(x_1,x_2) \rightarrow \sim(\exists x_3)\, A_1^1(x_3)) \rightarrow (\forall x_4)(\forall x_5)\, A_1^2(x_4,x_5)$



Prenex form

- Example:
  - change $((\forall x_1)\, A_1^2(x_1,x_2) \rightarrow \sim(\exists x_2)\, A_1^1(x_2)) \rightarrow (\forall x_1)(\forall x_2)\, A_1^2(x_1,x_2)$ to prenex form
  - first change the bound variables so that they are different from one another and from free variables:
  - $((\forall x_1)\, A_1^2(x_1,x_2) \rightarrow \sim(\exists x_3)\, A_1^1(x_3)) \rightarrow (\forall x_4)(\forall x_5)\, A_1^2(x_4,x_5)$
  - $((\forall x_1)\, A_1^2(x_1,x_2) \rightarrow (\forall x_3)\, \sim A_1^1(x_3)) \rightarrow (\forall x_4)(\forall x_5)\, A_1^2(x_4,x_5)$
  - $((\forall x_3)\, ((\forall x_1)\, A_1^2(x_1,x_2) \rightarrow \sim A_1^1(x_3))) \rightarrow (\forall x_4)(\forall x_5)\, A_1^2(x_4,x_5)$
  - $((\forall x_3)(\exists x_1)(A_1^2(x_1,x_2) \rightarrow \sim A_1^1(x_3)) \rightarrow (\forall x_4)(\forall x_5)\, A_1^2(x_4,x_5))$
  - $(\exists x_3)(\forall x_1)((A_1^2(x_1,x_2) \rightarrow \sim A_1^1(x_3)) \rightarrow (\forall x_4)(\forall x_5)\, A_1^2(x_4,x_5))$
  - $(\exists x_3)(\forall x_1)(\forall x_4)(\forall x_5)((A_1^2(x_1,x_2) \rightarrow \sim A_1^1(x_3)) \rightarrow A_1^2(x_4,x_5))$

  - *(sometimes answers are not unique)*



Π_n- form and Σ_n-form

- A wf. in prenex form is a Π_n- form if it starts with a universal quantifier and has n-1 alternations of quantifiers
- A wf. in prenex form is a Σ_n- form if it starts with an existential quantifier and has n-1 alternations of quantifiers

- $(\exists x_3)(\forall x_1)(\forall x_4)(\forall x_5)((A_1^2(x_1,x_2) \rightarrow \sim A_1^1(x_3)) \rightarrow A_1^2(x_4,x_5))$
  Σ_2- form



TQBF

- Instance: a fully quantified Boolean formula ϕ
  - We have seen ϕ can be written in prenex form
- Problem: to decide if ϕ is true

Example: a fully quantified Boolean formula

$$\forall x \exists y \forall z [(x \vee \sim y \vee z) \wedge (\sim x \vee y)]$$

Variables' range is {0,1}



TQBF is in PSPACE

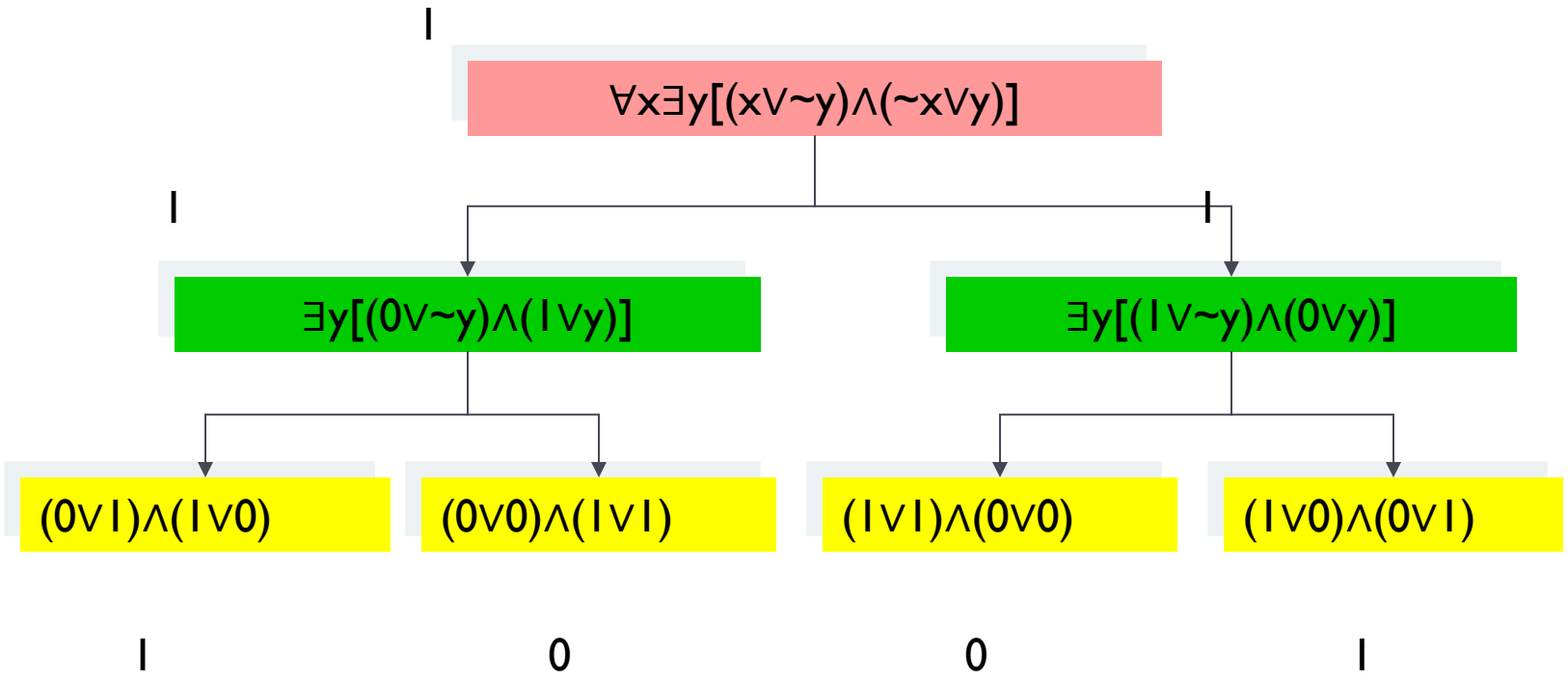
Theorem: TQBF∈PSPACE

Proof: We'll describe a poly-space algorithm A for evaluating ϕ

- If ϕ has no quantifiers: evaluate it
- If $\phi=\forall x(\psi(x))$ call A on $\psi(0)$ and on $\psi(1)$; Accept if *both* are true
- If $\phi=\exists x(\psi(x))$ call A on $\psi(0)$ and on $\psi(1)$; Accept if *either* is true



Algorithm for TQBF



Like a max-min tree



Efficiency

- Since both recursive calls use the same space,
- the total space needed is polynomial in the number of variables (the depth of the recursion)

⇒ TQBF is polynomial-space decidable ■



PSPACE Completeness

Definition:

A language B is PSPACE-Complete if

1. $B \in PSPACE$
2. For every $A \in PSPACE$, $A \leq_p B$

standard Karp reduction

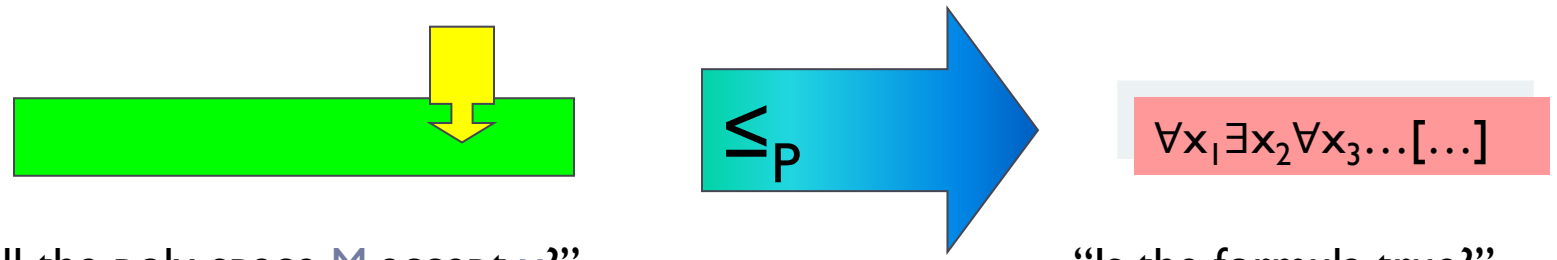
If (2) holds, then B is PSPACE-hard



TQBF is PSPACE-Complete

Theorem: TQBF is PSPACE-Complete

Proof: It remains to show TQBF is PSPACE-hard:





TQBF is PSPACE-Hard

- Given a poly-space TM M for a language $L \in$ PSPACE, and an input x,
  we construct a QBF ψ of polynomial size s.t.

  ψ is true iff M accepts x



TQBF is PSPACE-Hard

- Given a poly-space TM M for a language $L \in$ PSPACE, and an input x,
  we construct a QBF ψ of polynomial size s.t.

  ψ is true iff M accepts x

- In the TM M, for configurations u,v and an integer t, define $\psi(u,v,t)$=true iff v is reachable from u within t steps
  - $\psi(u,v,1)$ is efficiently computable
  - Similar to the proof of NP-hardness of SAT

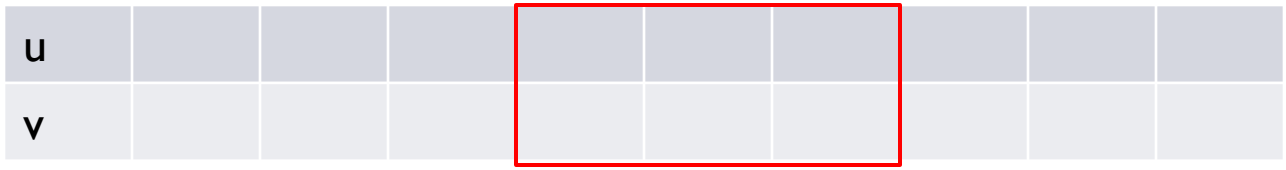


TQBF is PSPACE-Hard

- Given a poly-space TM M for a language L∈ PSPACE, and an input x,
  we construct a QBF ψ of polynomial size s.t.

  ψ is true iff M accepts x

- In the TM M, for configurations u,v and an integer t, define ψ(u,v,t)=true iff v is reachable from u within t steps
  - ψ(u,v,1) is efficiently computable
  - ψ(u,v,1) can be written as a polynomial sized formula for u and v

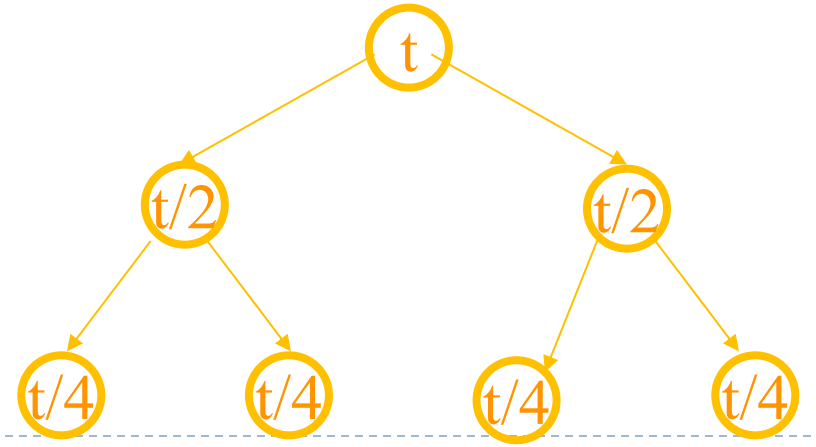


a 2x3 window is legal for transition, other variables are the same



- For configurations u,v and an integer t, define $\psi(u,v,t)$=true iff v is reachable from u within t steps

- For t>1, $\psi(u,v,t)=(\exists m)[\psi(u,m,t/2)\wedge\psi(m,v,t/2)]$
  - m is another configuration
  - Recursively deal with $\psi(u,m,t/2)\wedge\psi(m,v,t/2)$ until t=1
  - What's the problem?





- For configurations u,v and an integer t, define $\psi(u,v,t)$=true iff v is reachable from u within t steps

- For t>1, $\psi(u,v,t)=(\exists m)[\psi(u,m,t/2)\wedge\psi(m,v,t/2)]$
  - m is another configuration
  - the size of $\psi(u,v,t)$ is linear to t, but t is not polynomial as input x
  - Our machine M is PSPACE, not PTIME!
  - How to deal with that?



- For configurations u,v and an integer t, define $\psi(u,v,t)=true$ iff v is reachable from u within t steps

- For $t>1$, $\psi(u,v,t)=(\exists m)[\psi(u,m,t/2)\wedge\psi(m,v,t/2)]$
  - m is another configuration

- Change to:

$$\psi(u,v,t)=(\exists m)(\forall(a,b)\in\{(u,m),(m,v)\})[\psi(a,b,t/2)]$$



- For configurations u,v and an integer t, define $\psi(u,v,t)=true$ iff v is reachable from u within t steps.

- For $t>1$, $\psi(u,v,t)=(\exists m)[\psi(u,m,t/2)\wedge\psi(m,v,t/2)]$
  - m is another configuration

- Change to:

  $\psi(u,v,t)=(\exists m)(\forall (a,b)\in\{(u,m),(m,v)\})[\psi(a,b,t/2)]$

  which is:

  $\psi(u,v,t)=(\exists m)(\forall a)(\forall b)[((a=u\wedge b=m)\vee(a=m\wedge b=v))\rightarrow\psi(a,b,t/2)]$

size: $S(t)=S(t/2)+$(polynomial of $|x|$),
so $S(t)$ is polynomial of $\log t$ and $|x|$, which is polynomial of $|x|$
(t is exponential, which needs polynomial number of bits)



TQBF is PSPACE-Complete

Claim: TQBF is PSPACE-Complete

Proof:

$\psi_{M,x} \equiv \psi(c_{start}, c_{accept}, h)$ is TRUE iff M accepts x in t steps

ψ is constructible in poly-time (in prenex form)

Thus, any PSPACE language is poly-time reducible to TQBF, i.e – TQBF is PSPACE-hard

Since TQBF$\in$PSPACE, it's PSPACE-Complete ■



Formula Game

- two players A and E,
- a Boolean formula ψ with variables $x_1, x_2, \ldots, x_n$

- The game:
  - A selects the value of x_1
  - E selects the value of x_2
  - A selects the value of x_3
  - E selects the value of x_4
  - …
- Finally, A wins if ψ is false, E wins if ψ is true

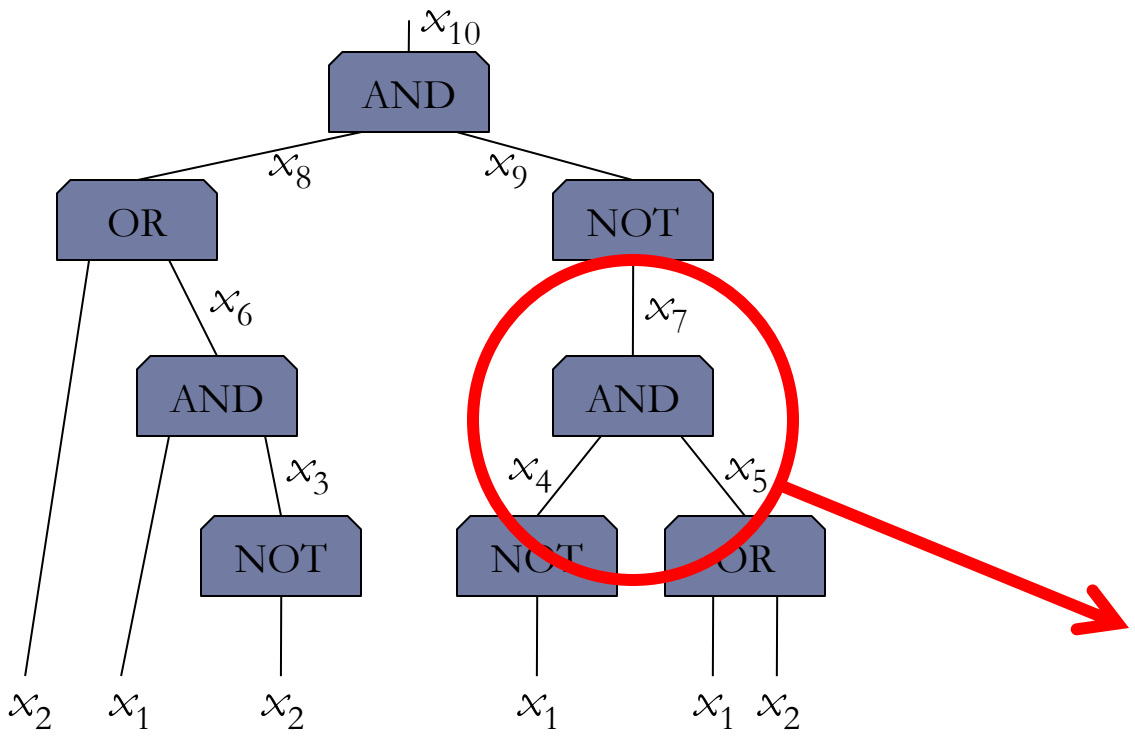
- E has winning strategy iff $(\forall x_1)(\exists x_2)(\forall x_3)(\exists x_4)\ldots[\psi]$ is true
  - Thus, the formula game is equivalent to a TQBF



- Is the Formula Game $(\forall x_1)(\exists x_2)(\forall x_3)(\exists x_4)\ldots[\psi]$ PSPACE-complete if ψ is a 3-CNF?



- The Formula Game $(\forall x_1)(\exists x_2)(\forall x_3)(\exists x_4)\ldots[\psi]$ PSPACE-complete even if ψ is a 3-CNF
  - Example: $\psi = (x_2 \vee (x_1 \wedge \overline{x_2})) \wedge \overline{(\overline{x_1} \wedge (x_1 \vee x_2))}$



x_4	x_5	x_7	$x_7 = x_4 \wedge x_5$	
T	T	T	T	
T	T	F	F	$\leftarrow (\overline{x_4} \vee \overline{x_5} \vee x_7)$
T	F	T	F	$\leftarrow (\overline{x_4} \vee x_5 \vee \overline{x_7})$
T	F	F	T	
F	T	T	F	$\leftarrow (x_4 \vee \overline{x_5} \vee \overline{x_7})$
F	T	F	T	
F	F	T	F	$\leftarrow (x_4 \vee x_5 \vee \overline{x_7})$
F	F	F	T	

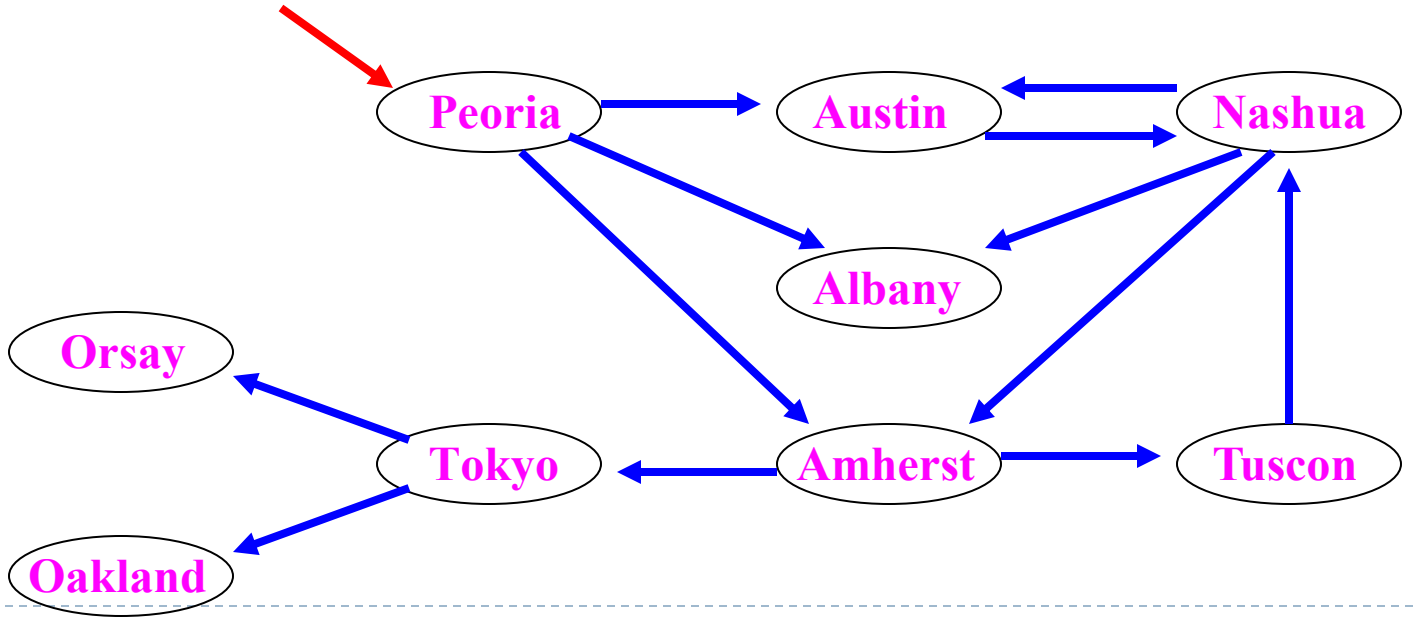
$$(\overline{x_4} \vee \overline{x_5} \vee x_7) \wedge (\overline{x_4} \vee x_5 \vee \overline{x_7}) \wedge (x_4 \vee \overline{x_5} \vee \overline{x_7}) \wedge (x_4 \vee x_5 \vee \overline{x_7})$$

We give an extra variable to every gate ("wire")

$\psi(x)$ is true $\Leftrightarrow$ $(\exists y)\phi(x,y)$ is true
$\phi(x,y)$ is a 3-CNF

Generalized geography game (接龙游戏)

- Players 1 and 2 take turns naming cities from anywhere in the world (player 1 starts). Each city chosen must begin with the same letter that ended the previous city's name. The player who is unable to continue loses.





- But no repetition is allowed



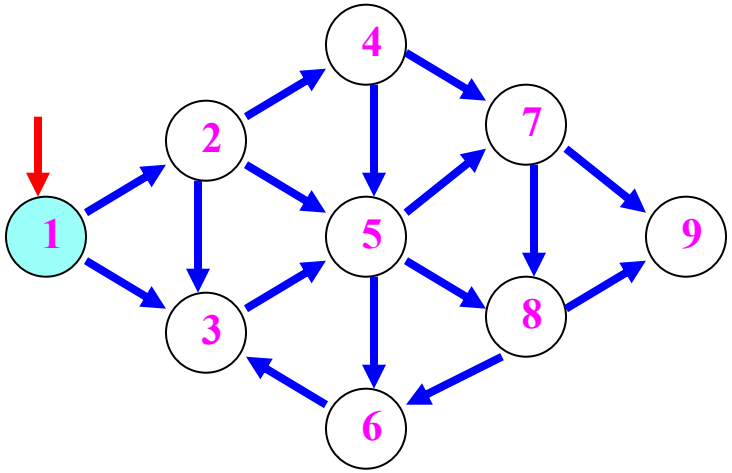
✅



❌

Generalized geography game

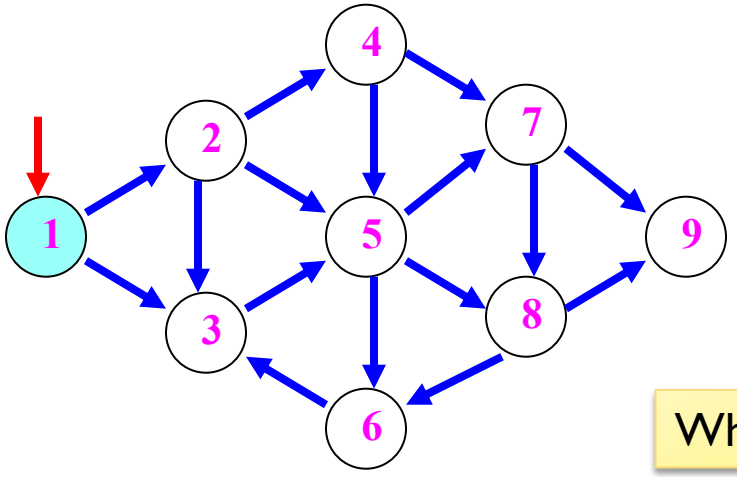
- In a directed graph with a start node s
  - Initially token is at s
  - Player 1 and 2 alternatively choose the next node, if the token is currently at v, it can choose w if (v,w) is an edge
  - No repetition is allowed
  - When a player cannot move on, it loses





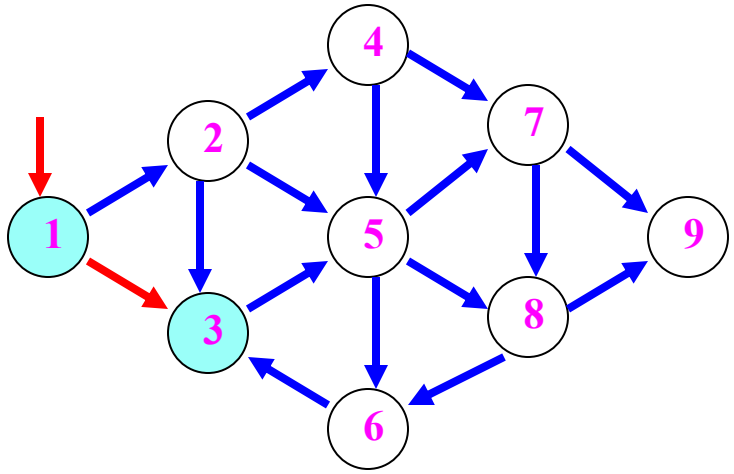
Generalized geography game

- In a directed graph with a start node s
  - Initially token is at s.
  - Player 1 and 2 alternatively choose the next node, if the token is currently at v, it can choose w if (v,w) is an edge
  - No repetition is allowed
  - When a player cannot move on, it loses



Who has a winning strategy here?

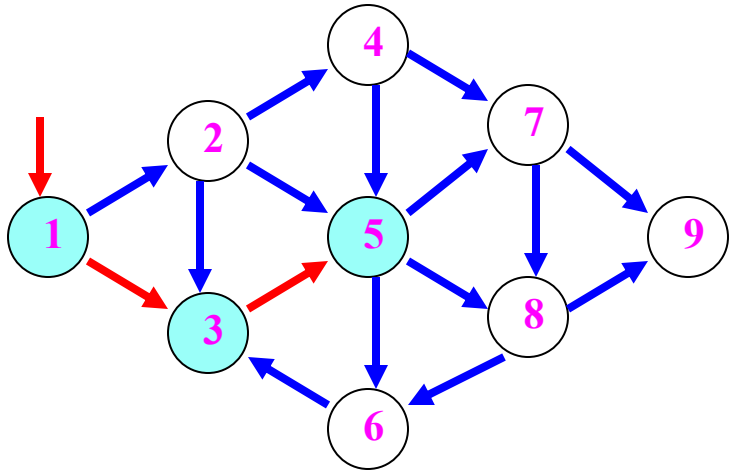




Who has a winning strategy here?

Player I: Choose 3.

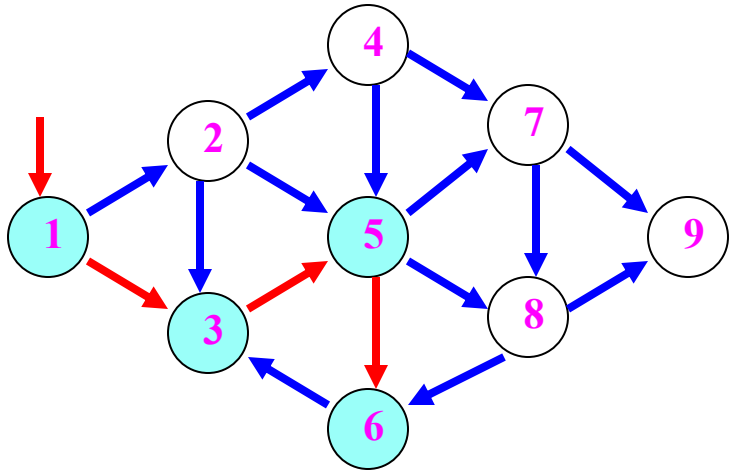




Who has a winning strategy here?

Player I: Choose 3. II will have to choose 5.





Who has a winning strategy here?

Player I: Choose 3. II will have to choose 5.
Now I selects 6, and II is stuck.

Generalized geography game

GG = {<G,b> | Player I has a winning strategy for the Generalized Geography game played on graph G starting at node b}

- GG is PSPACE-complete

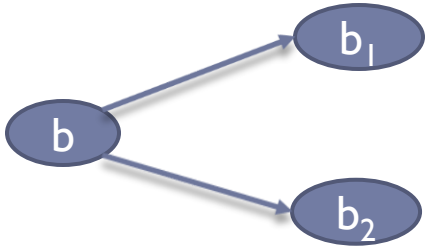
A recursive algorithm similar to the one used for TQBF can show that GG is in PSPACE



Generalized geography game

GG = {<G,b> | Player **I** has a winning strategy for the Generalized Geography game played on graph **G** starting at node **b}**

- GG is in PSPACE:



Recursively consider:
<G\{b}, b_1> and <G\{b}, b_2>

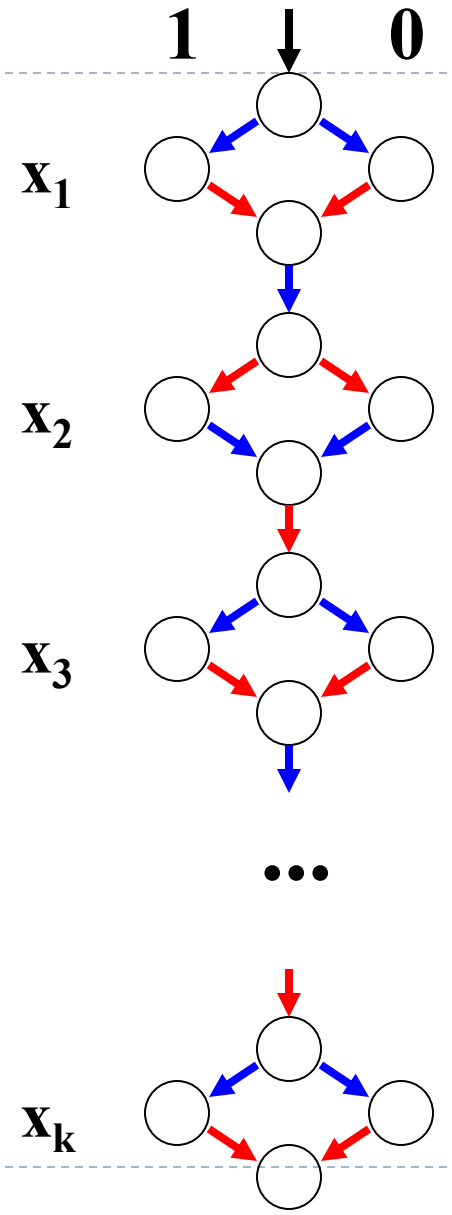


GG is PSPACE-hard

- Reduce the Formula Game for 3CNF to GG game
  - (w.l.o.g. also assume the first and last quantifiers are ∃)
  - $(\exists x_1)(\forall x_2)(\exists x_3)\ldots(\exists x_k)[c_1 \wedge c_2 \wedge \ldots c_m]$, where each $\mathbf{c_i}$ is a disjunction of three literals



Why GG is PSPACE-hard



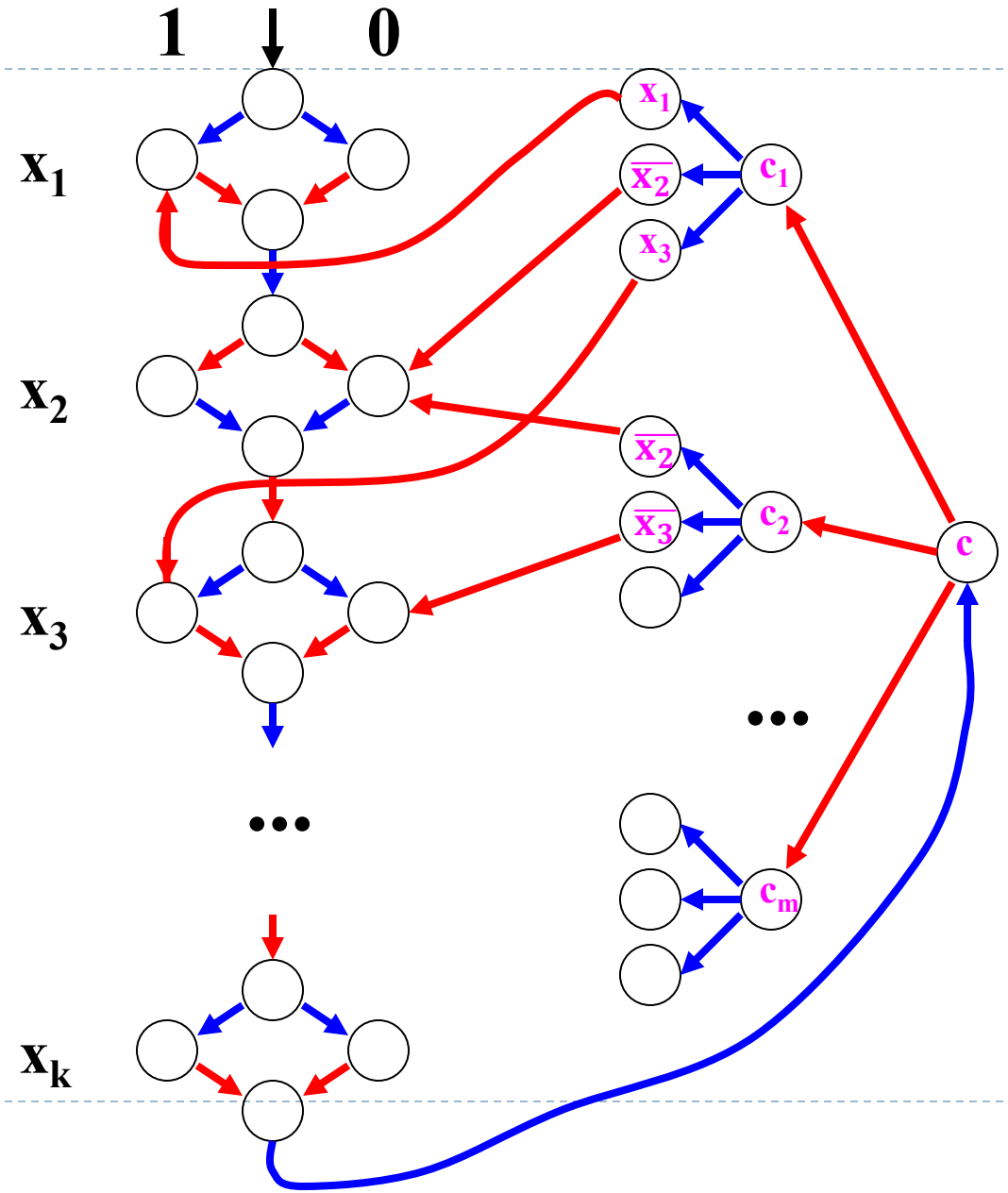
The left part of **G** will look like this: for each variable we create a diamond. The blue arrows indicate **Player I**'s choices (turns), and the red arrows indicate **Player II**'s choices in the game.

The left-hand choices will correspond to choosing **1** in the formula game, and the right-hand choices correspond to choosing **0**.

Then we extend this graph by adding to it the right part as shown on the next slide for a particular example:

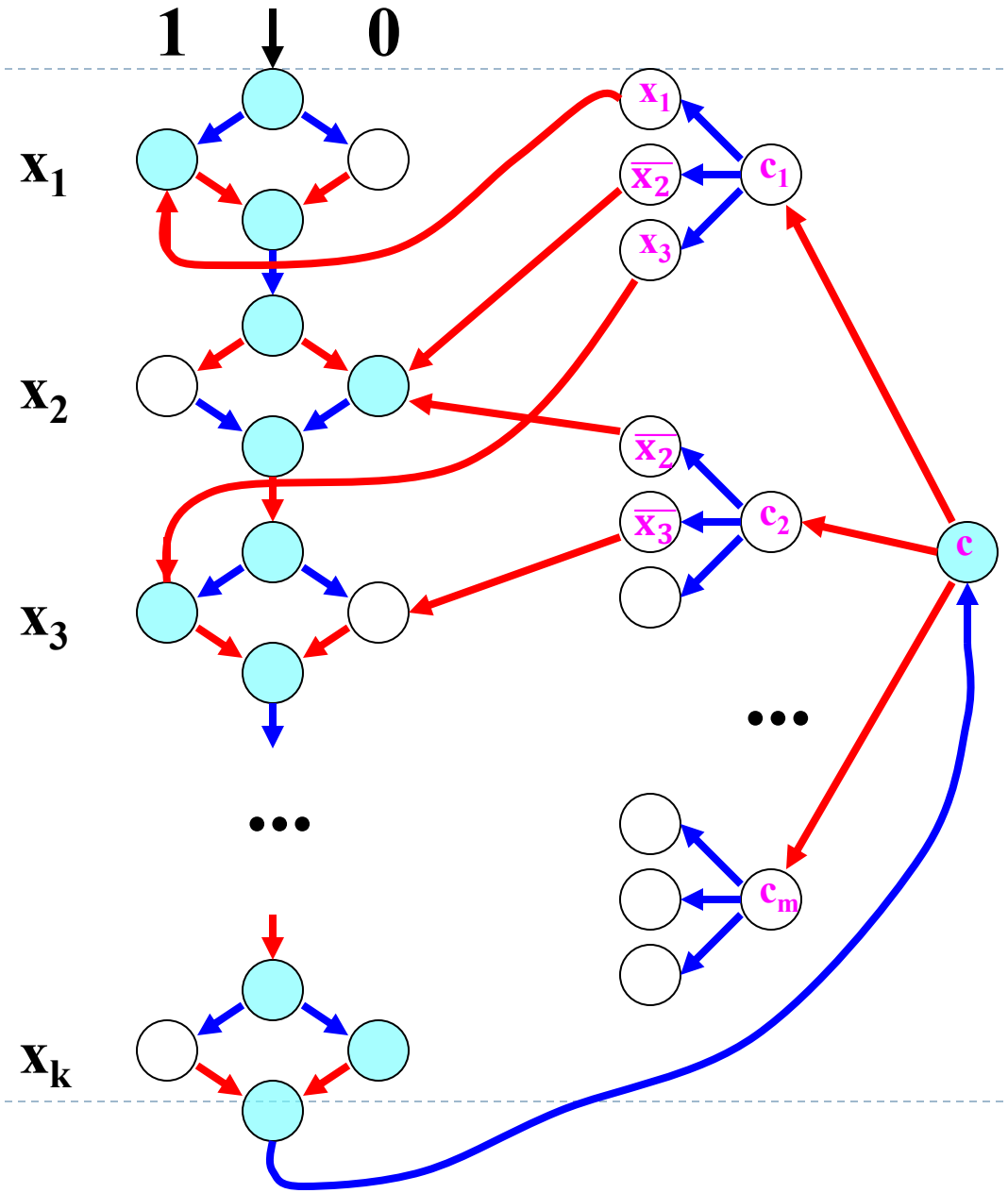
ϕ: $\exists x_1 \forall x_2 \exists x_3 \ldots \exists x_k [(x_1 \vee \sim x_2 \vee x_3) \wedge (\sim x_2 \vee \sim x_3 \vee \ldots) \wedge \ldots]$

ϕ: $\exists x_1 \forall x_2 \exists x_3 ... \exists x_k [(x_1 \vee \sim x_2 \vee x_3) \wedge (\sim x_2 \vee \sim x_3 \vee ...) \wedge ...]$



Suppose **E** has a winning strategy in ϕ.
(ϕ can be satisfied.)

ϕ: $\exists x_1 \forall x_2 \exists x_3 \ldots \exists x_k [(x_1 \vee \sim x_2 \vee x_3) \wedge (\sim x_2 \vee \sim x_3 \vee \ldots) \wedge \ldots]$



Suppose **E** has a winning strategy in ϕ. (ϕ can be satisfied.)

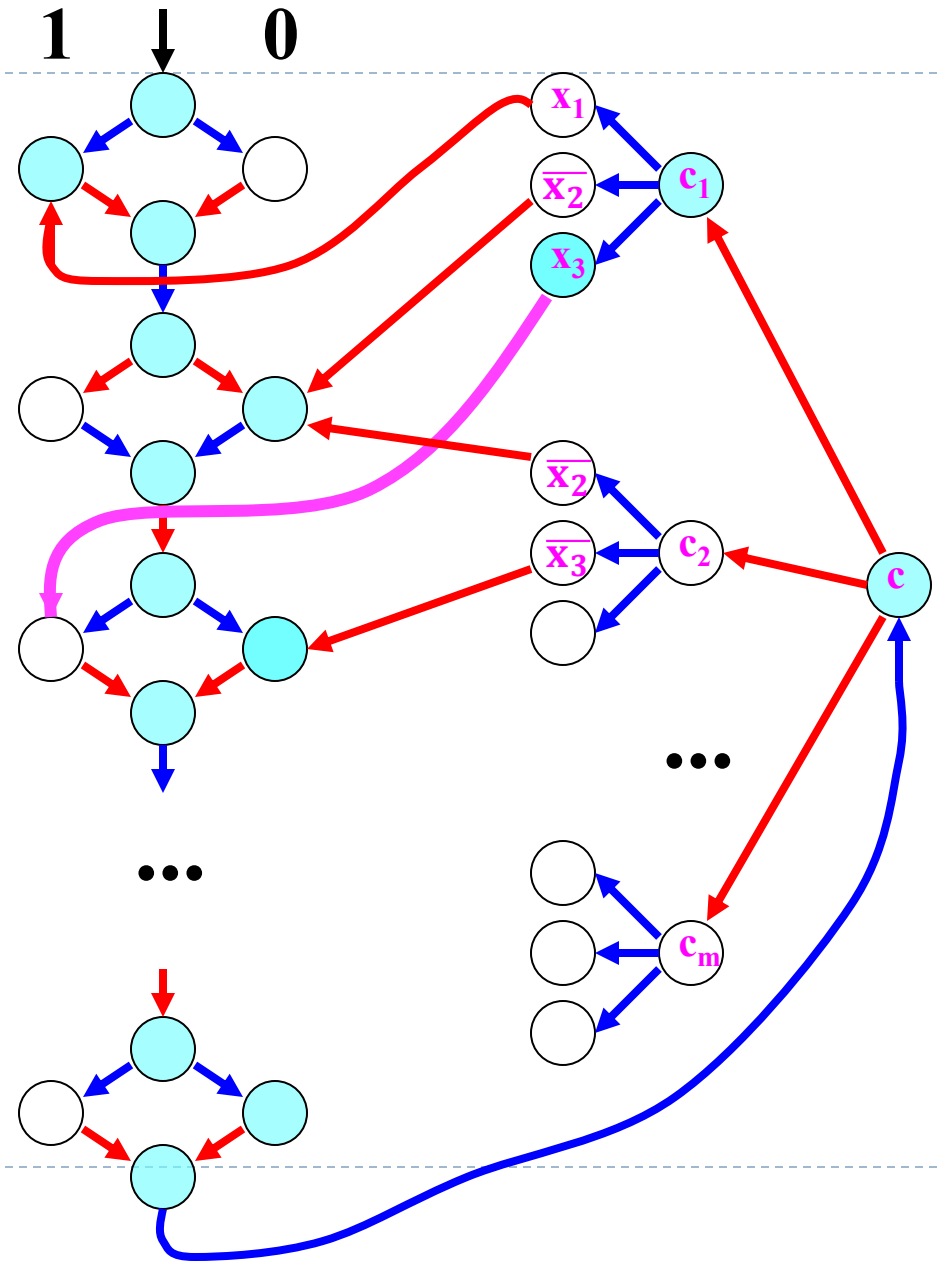
Whatever $\mathbf{c_i}$ is chosen by Player II, it is a true clause and thus has a true literal.

Let then Player I choose such a true literal.

Then Player II is stuck, as the path has already passed through the corresponding left or right node of the corresponding diamond.

So, Player I has a winning strategy in **G**.

ϕ: $\exists x_1 \forall x_2 \exists x_3 ... \exists x_k [(x_1 \vee \sim x_2 \vee x_3) \wedge (\sim x_2 \vee \sim x_3 \vee ...) \wedge ...]$



Suppose now **A** has a winning strategy in ϕ. (ϕ cannot be satisfied.)

Then there is a false clause c_i, and let Player II choose that clause.

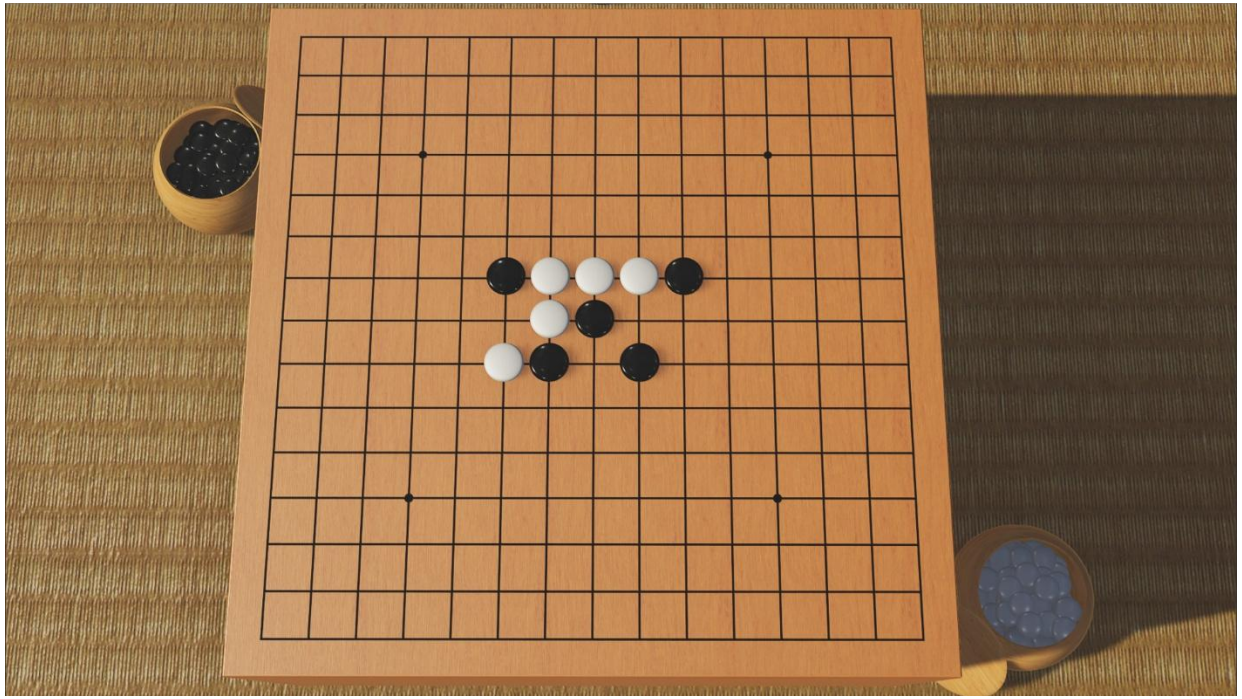
Now, whatever literal of c_i is chosen by Player I, it is a false literal and hence the path has not passed through it.

So, Player II is not stuck, and goes to the corresponding left or right node of the corresponding diamond.

Now Player I is stuck. Thus, II has a winning strategy in **G**.

Gomoku (five-in-a-row)

- Players alternate turns placing a stone of their color on an empty intersection. Black plays first. The winner is the first player to form an unbroken line of five stones of their color horizontally, vertically, or diagonally



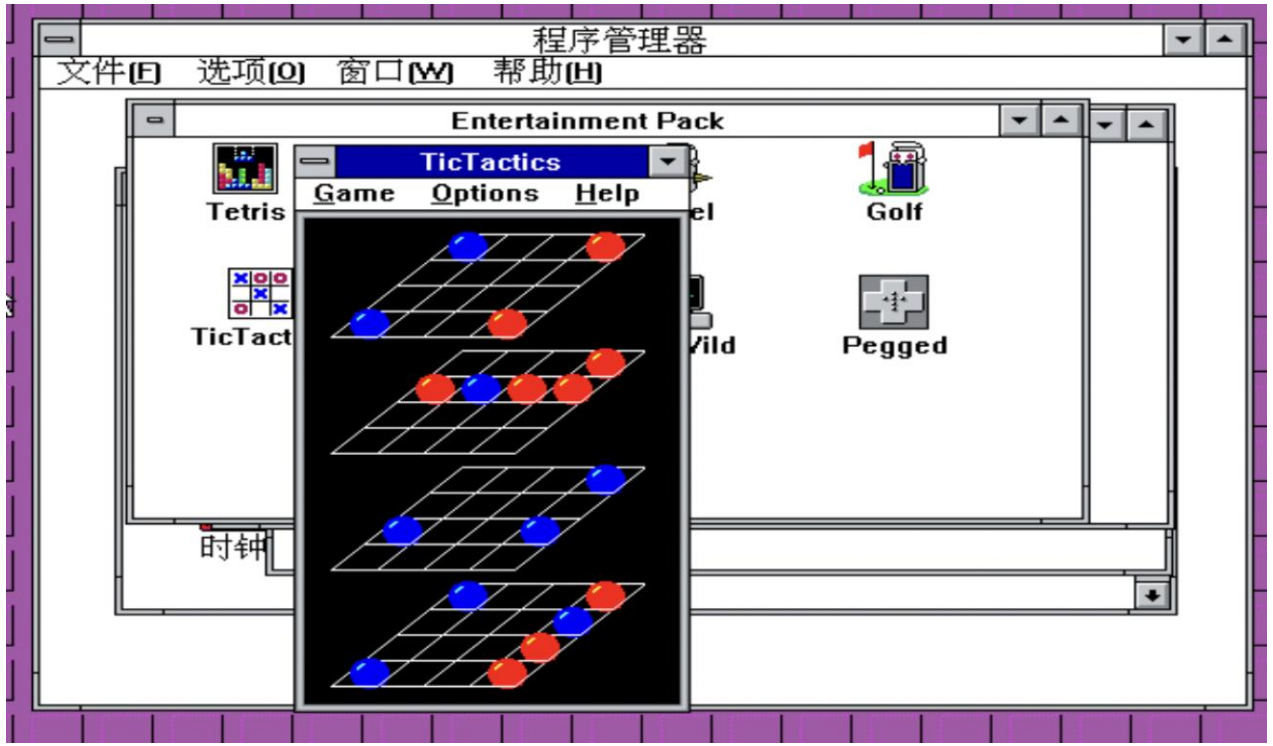


Gomoku (five-in-a-row)

- Players alternate turns placing a stone of their color on an empty intersection. Black plays first. The winner is the first player to form an unbroken line of five stones of their color horizontally, vertically, or diagonally

- Generalizes tic-tac-toe:



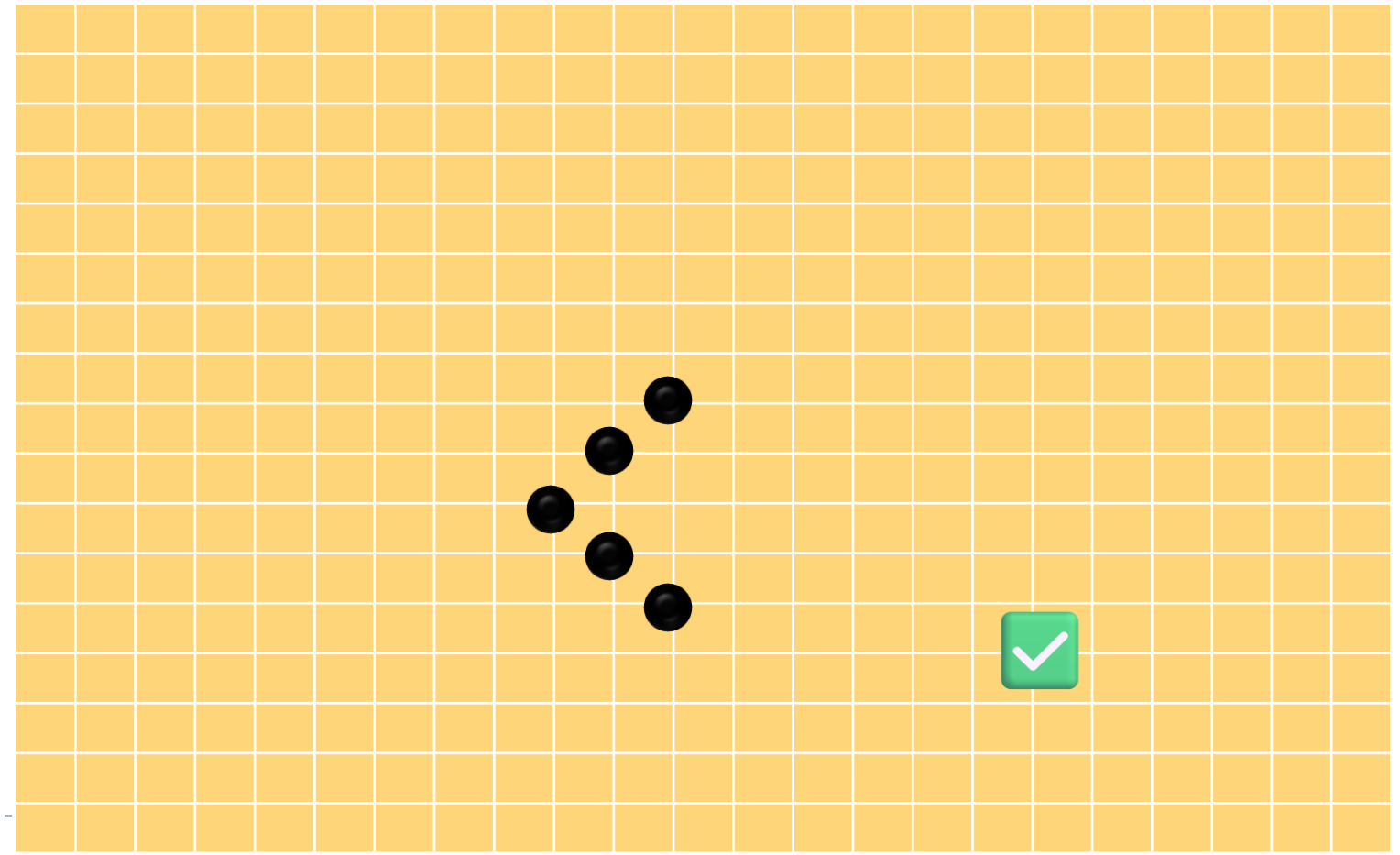




Gomoku is PSPACE-complete

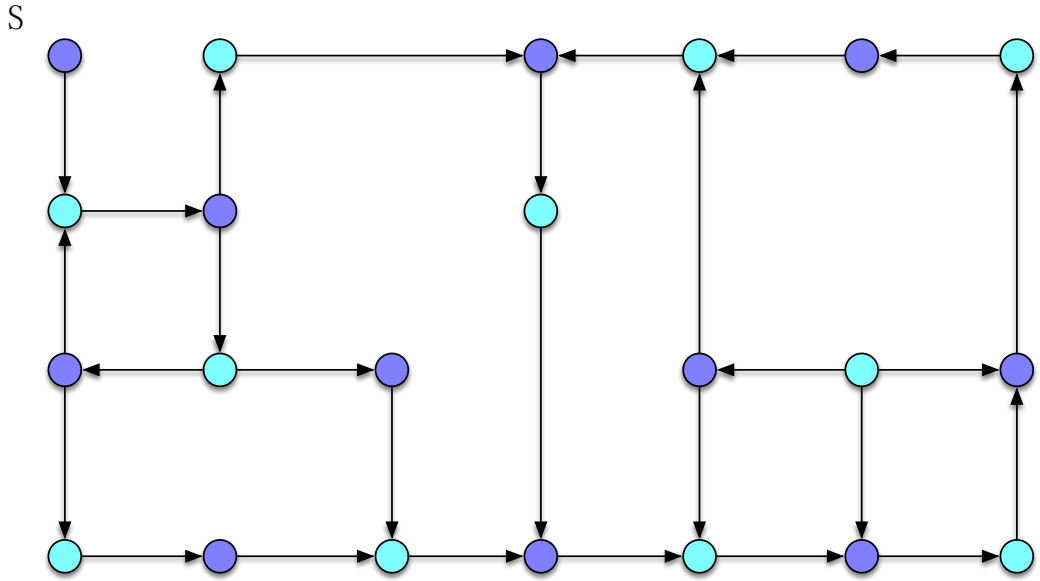
- *[Stefan Reisch 1980]* "Gobang ist PSPACE-vollständig"

- Generalized version:
  - played on n × n board
  - starting at any situation
  - no *forbidden moves (禁手)*



Reduction from restricted version of GG game

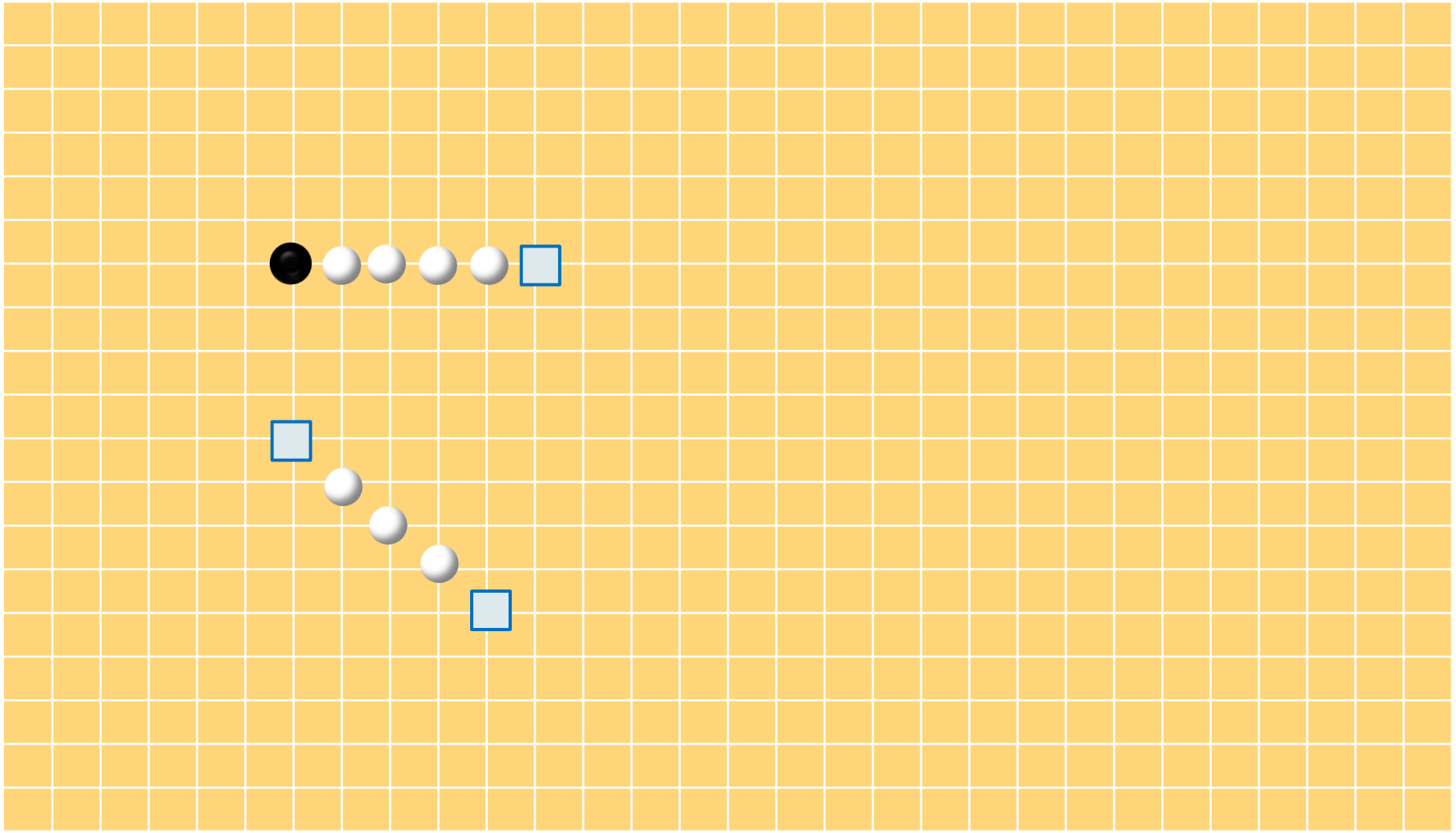
- Generalized geography game is still PSPACE-complete on grid-graphs:
  - graph is a grid-graph and bipartite
  - starting vertex s has indegree 0 and outdegree 1
  - other vertices has total degree ≤3, and indegree>0, outdegree>0
  - each edge is either of length 1 or length 2
  - edges going out from the same vertex make an angle of 180° with each other





a “threat”:

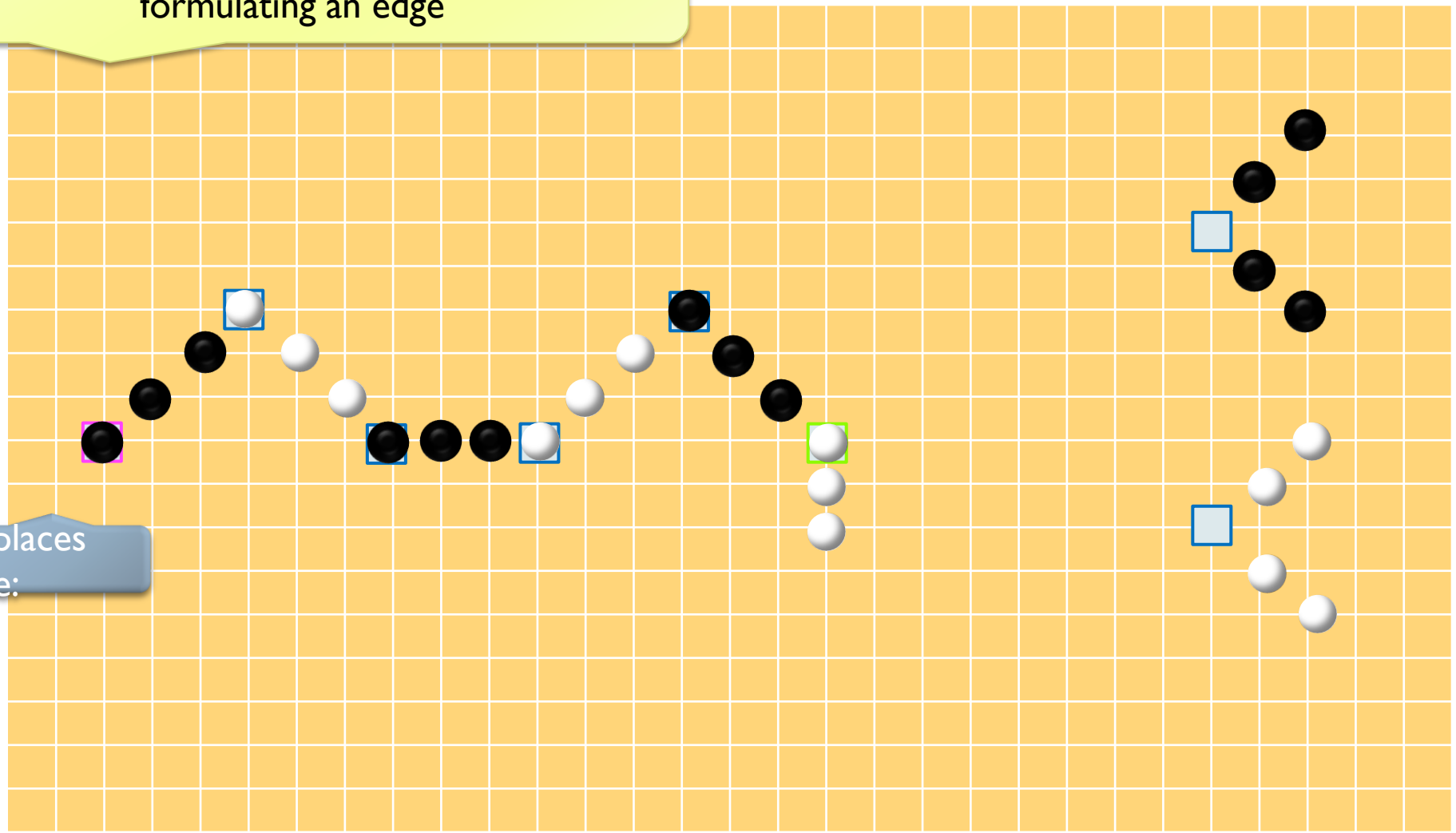
- row of 4 with one open end
- row of 3 with two open ends





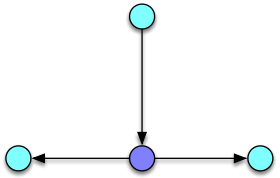
In the game, each player needs to make a threat in every step, otherwise the other player wins

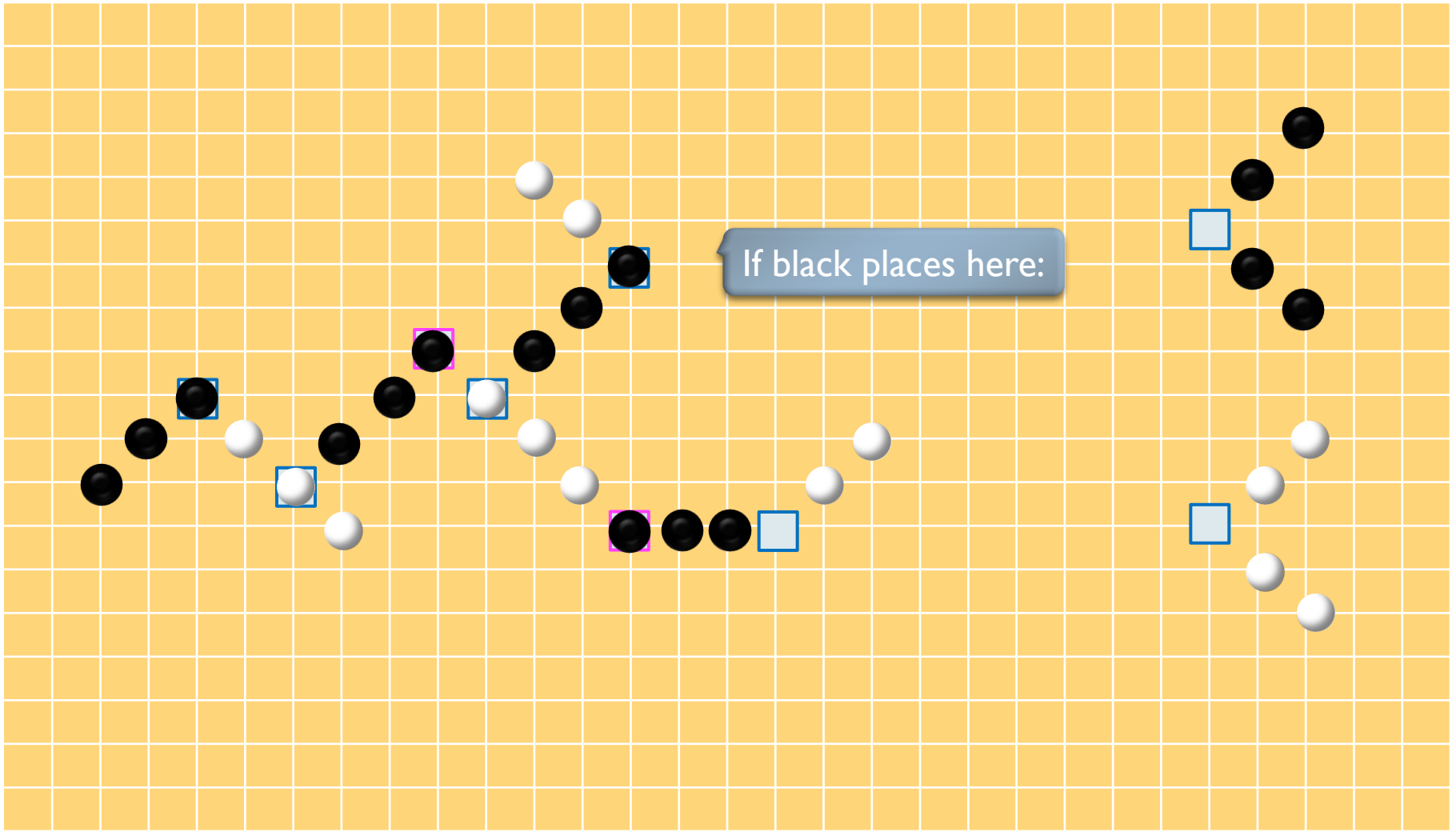
This chain is a pattern of forced move, formulating an edge



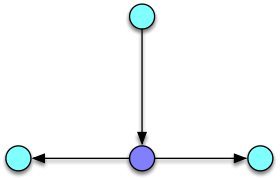
If black places here:

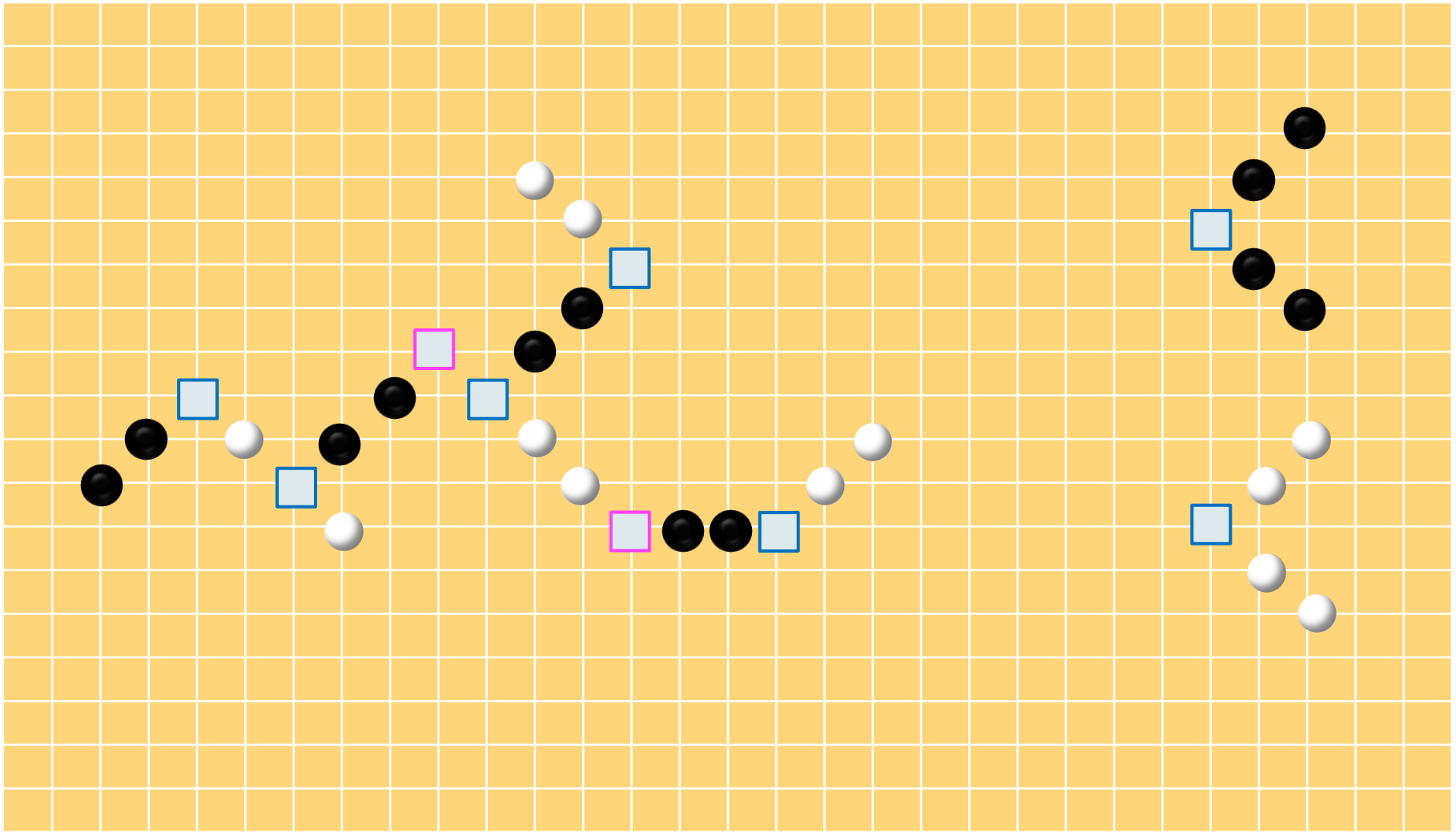
Formulating:





Formulating:

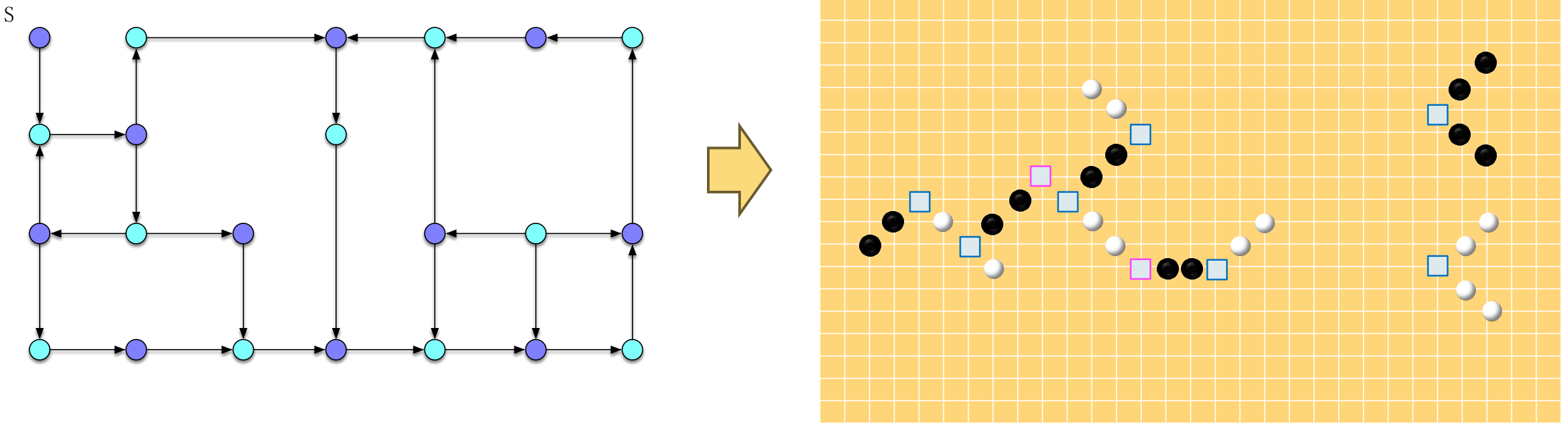






Reduction from restricted GG game to generalized Gomoku

- So n × n Gomoku is PSPACE-complete



Summary



- Our main results were:
  - PATH is NL-Complete
  - Savitch's theorem (NL$\subseteq$SPACE($\log^2 n$))
  - The padding argument (extending results for space complexity)
  - Immerman's theorem (NL=coNL)
  - PSPACE-Completeness

